



universität
uulm

**Fakultät für
Ingenieurwissenschaften,
Informatik und
Psychologie**
Institut für Datenbanken
und Informationssysteme (DBIS)

Konzeption und Implementierung einer webbasierten Umgebung für die Erstellung und Ausführung von gamifizierten BPMN 2.0 Modellen

Bachelorarbeit an der Universität Ulm

Vorgelegt von:

Merten Dieckmann
merten.dieckmanne@uni-ulm.de
1058340

Gutachter:

Prof. Dr. Manfred Reichert

Betreuer:

Michael Winter

2023

Fassung 15. Juni 2023

© 2023 Merten Dieckmann

Satz: PDF-L^AT_EX 2_ε

Zusammenfassung

Durch die fortschreitende Digitalisierung der Arbeitswelt ist die Notwendigkeit einer effektiven Geschäftsprozessmodellierung unbestreitbar. Insbesondere die Verwendung von Business Process Model and Notation (BPMN) 2.0 hat sich als Standard in der Industrie etabliert. Geschäftsprozesse werden jedoch von den Mitarbeitern oft als monoton und wenig motivierend wahrgenommen. Diese Arbeit zeigt auf, wie diesem Problem durch die Integration von Gamifizierungselementen – speziell Punkten, Auszeichnungen, Herausforderungen und einer Minimap – in BPMN 2.0 Modelle entgegengewirkt werden kann. Dafür wurde eine benutzerfreundliche webbasierte Umgebung für die Modellierung und Ausführen von gamifizierten Geschäftsprozessen konzipiert und mit React.js und TypeScript implementiert. Nach der Implementierung erfolgte eine detaillierte Analyse der funktionalen und nicht-funktionalen Anforderungen und erste Nutzertests zur Bewertung der Effektivität der spielerischen Elemente wurden durchgeführt. Diese Nutzertests konnten erste Tendenzen aufzeigen, dass die Gamifizierungselemente zu einem erhöhten Engagement der Benutzer in den Prozessen beitragen können. Allerdings muss die langfristige Auswirkung der Elemente auf die Motivation noch durch umfangreichere Studien erforscht werden. Die Ergebnisse dieser Arbeit weisen darauf hin, dass die Integration von Gamifizierung in BPMN 2.0 Modelle ein vielversprechender Ansatz zur Steigerung der Benutzerbeteiligung und Motivation ist und bietet einen Ausblick auf zukünftige Forschungsrichtungen.

Inhaltsverzeichnis

1	Einführung	1
1.1	Motivation	1
1.2	Ziele der Bachelorarbeit	2
1.3	Struktur der Arbeit	2
2	Grundlagen von BPMN 2.0	3
2.1	Aktivitäten	4
2.2	Sequenzfluss	4
2.3	Events	5
2.4	Gateways	5
2.5	Datenobjekte und Assoziationen	6
2.6	BPMN 2.0 XML	6
3	Verwandte Arbeiten	7
3.1	Gamifizierung	7
3.1.1	Punkte	9
3.1.2	Auszeichnungen	9
3.1.3	Herausforderungen	10
3.1.4	Minimap	10
3.2	Erweiterung des BPMN-Modells	11
3.3	Abgrenzung zu anderen Arbeiten	12
4	Anforderungsanalyse	13
4.1	Funktionale Anforderungen an den Editor	14
4.2	Funktionale Anforderungen an die Engine	17
4.3	Nichtfunktionale Anforderungen	19

5	Design und Funktionalitäten	20
5.1	Editor	20
5.1.1	Start	21
5.1.2	Ende	21
5.1.3	Aktivität	22
5.1.4	Gateway	26
5.1.5	Herausforderung	27
5.1.6	Info	28
5.1.7	Gamifizierungsevent	29
5.2	Engine	29
5.2.1	Aufgabenfenster	30
5.2.2	Belohnungen	31
5.2.3	Herausforderungen	32
5.2.4	Minimap	32
5.3	Modelltransformation	33
6	Implementierung	35
6.1	Grundlagen der Implementierung	35
6.1.1	JavaScript	35
6.1.2	TypeScript	36
6.1.3	Polymorphismus	36
6.1.4	JSON	37
6.1.5	React.js	37
6.1.6	Create-React-App	37
6.1.7	React Flow	38
6.1.8	Zustand	38
6.1.9	Weitere Bibliotheken	39
6.2	Editor	39
6.2.1	React Flow und JSON-Format	40
6.2.2	Komponenten zur Repräsentation von Flow-Elementen	40
6.3	Engine	41
6.3.1	Prozess-Import und Engine-Architektur	41
6.3.2	Prozessausführung	42
6.3.3	Speichern und Anzeigen von Gamifizierungselementen	45

7	Ergebnisse und Diskussion	46
7.1	Vergleich funktionaler Anforderungen	46
7.2	Vergleich nichtfunktionaler Anforderungen	47
7.2.1	Benutzerfreundlichkeit	47
7.2.2	Wartbarkeit	49
7.2.3	Portabilität	50
7.3	Effektivität der Gamifizierungselemente	50
7.3.1	Nutzertests	50
7.3.2	Langzeitmotivation	51
7.3.3	Vorläufige Schlussfolgerungen	51
8	Aussicht und Zusammenfassung	53
	Literatur	54
A	Quelltexte	57

1 Einführung

Dieses Kapitel leitet thematisch in die Arbeit ein und gibt einen Überblick über die Ziele und die Struktur.

1.1 Motivation

Die fortschreitende Digitalisierung hat in den vergangenen Jahren die Arbeitswelt tiefgreifend verändert. Traditionelle Arbeitsweisen werden immer mehr durch digitale Prozesse ersetzt. Insbesondere die Modellierung von Geschäftsprozessen mithilfe der Business Process Model and Notation (BPMN) hat sich dabei, aufgrund seiner einfachen Handhabung und universellen Einsetzbarkeit, als Standard durchgesetzt [1]. BPMN ist eine grafische Darstellung für die Spezifikation von Geschäftsprozessen [19].

Eine wesentliche Herausforderung stellt jedoch dabei die Einbindung und Motivation der Benutzer dar. Durch die oft technische und abstrakte Darstellung können Geschäftsprozesse als monoton und wenig motivierend wahrgenommen werden. Diese Wahrnehmung kann zu einer negativen Auswirkung auf die Effizienz und Produktivität der Geschäftsprozesse führen. Die Idee, diese Prozesse um spielerische Elemente zu erweitern, bietet eine vielversprechende Möglichkeit, diese negativen Auswirkungen zu mindern und sowohl die Benutzerfreundlichkeit, als auch die Motivation bei der Bearbeitung zu erhöhen [11].

1.2 Ziele der Bachelorarbeit

Das Hauptziel dieser Arbeit ist die Konzeption und Implementierung einer webbasierten Umgebung, die die Modellierung und Ausführung von gamifizierten Geschäftsprozessen ermöglicht. Dafür werden basierend auf aktuellen Forschungsergebnissen zur Gamifizierung und Motivation spezifische Gamifizierungselemente vorgestellt, mit welchen das BPMN 2.0 Modell erweitert werden kann. Im weiteren Verlauf der Arbeit wird auf der Grundlage dieser Elemente eine webbasierte Umgebung zur Erstellung und Ausführung von gamifizierten BPMN 2.0 Modellen implementiert. Abschließend erfolgt eine Evaluierung der Effektivität dieser Umgebung in der Praxis und ein Ausblick auf zukünftige Forschungsarbeiten.

1.3 Struktur der Arbeit

Die Arbeit ist wie folgt strukturiert: Kapitel 2 führt die grundlegenden Konzepte von BPMN 2.0 ein. Kapitel 3 behandelt relevante Ergebnisse verwandter Werke zur Gamifizierung und zur Erweiterung von BPMN-Modellen und legt dar, inwiefern diese Bachelorarbeit diese Ergebnisse erweitert. Dort werden ebenfalls die Gamifizierungselemente vorgestellt, welche die webbasierte Umgebung unterstützt. In Kapitel 4 wird eine Anforderungsanalyse durchgeführt, die sowohl funktionale als auch nichtfunktionale Anforderungen für den zu entwickelnden Editor und die Engine festlegt. Kapitel 5 geht auf das Design und die Funktionalitäten des Editors und der Engine ein, und Kapitel 6 beschreibt deren Implementierung. Kapitel 7 diskutiert die Ergebnisse der Implementierung und bewertet die Effektivität der Gamifizierungselemente auf Basis von Nutzertests. Schließlich gibt Kapitel 8 einen Ausblick auf zukünftige Arbeiten und fasst die Ergebnisse zusammen.

2 Grundlagen von BPMN 2.0

Dieses Kapitel thematisiert die für diese Arbeit notwendigen Grundlagen von BPMN 2.0.

Business Process Model and Notation (BPMN) ist eine Standard-Notation, um Geschäftsprozesse zu modellieren, welche von allen Teilnehmern verstanden werden können. Die aktuelle Version 2.0 des Standards wurde 2011 von der Object Management Group (OMG) entwickelt. Prozesse bestehen aus einer Abfolge von Flow-Elementen. Diese Flow-Elemente sind durch Sequenzflüsse miteinander verbunden, welche den Ablauf des Prozesses vorgeben. Der Umfang von Prozessen kann von Aufgaben für eine Person hin zu firmenweiten Tätigkeiten reichen [19].

Ein Ziel der OMG ist es, eine Brücke zwischen der Modellierung und der Implementierung zu schaffen. Der Standard kann im Dialog zwischen Geschäftspartnern jeder Ebene als schnelle Anforderungsübersicht dienen, den Entwicklern helfen, die Geschäftsprozessen zu implementieren und im Anschluss das Überwachen und Warten erleichtern. Ein anderes Ziel von BPMN 2.0 ist es, Prozesse visualisieren zu können, welche durch eine zur Ausführung entworfene XML Sprache modelliert sind [19].

In der BPMN 2.0 Spezifikation werden drei Unterprozesstypen beschrieben: Konversationsdiagramme, Kollaborationsdiagramme und Prozessdiagramme. Konversationsdiagramme modellieren die Logik vom Nachrichtenaustausch. Kollaborationsdiagramme beschreiben die Interaktion von verschiedenen Prozessteilnehmern. Prozessdiagramme sind für den Prozessablauf von Aktivitäten da und werden in der BPMN als Graph dargestellt [19]. In dieser Arbeit wird mit Prozessdiagrammen gearbeitet. Nachfolgend folgt eine Erklärung aller für diese Arbeit relevanten Flow-Elemente.

2.1 Aktivitäten

Eine Aktivität ist eine Tätigkeit, welche innerhalb von einem Geschäftsprozess ausgeführt wird. Es gibt drei Arten von Aktivitäten: Aufgabe, Subprozess und das Aufrufen von einer anderen Aktivität. Eine Aufgabe ist eine kleinste mögliche atomare Aktivität, welche meist von einem Benutzer ausgeführt wird. Sie wird durch ein abgerundetes Rechteck abgebildet (siehe Abbildung 2.1). Eine Aufgabe kann zudem einen Typen zugewiesen bekommen. Dieser wird oben links im Rechteck durch ein passendes Symbol dargestellt. Diese Eigenschaft wird im weiteren Verlauf der Arbeit für die Gamifizierung noch interessant werden [19].



Abbildung 2.1: Aufgabe ohne und Aufgabe mit Typ

2.2 Sequenzfluss

Sequenzflüsse sind die Verbindung zwischen Aktivitäten und geben den Prozessablauf vor. Eine Aktivität hat immer genau einen ausgehenden Sequenzfluss und beliebig viele eingehende Sequenzflüsse. Hat eine Aktivität mehr als einen eingehenden Sequenzfluss, handelt es sich um einen unkontrollierten Ablauf. Die Aktivität wird ausgeführt, sobald über einen der Sequenzflüsse das Signal kommt [19]. Ein Sequenzfluss wird durch einen Pfeil mit gefüllter Spitze dargestellt (siehe Abbildung 2.2).

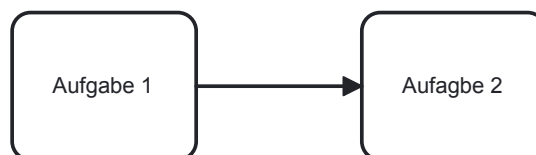


Abbildung 2.2: Sequenzfluss zwischen zwei Aufgaben

2.3 Events

Ein Event ist etwas, was während des Prozesses passieren kann. Es kann im Prozess durch etwas ausgelöst werden oder andere Sachen im Prozess auslösen. Es gibt drei Arten von Events. Das Start-Event gibt den Start des Prozesses an. Ein Ende-Event beendet den Prozess. Hiervon kann es mehrere im Prozess geben. Das Intermediate-Event findet während des Prozesses statt und kann den Prozess weder starten noch beenden. Es kann den Ablauf des Prozesses beeinflussen und kann unter anderem dafür benutzt werden aufzuzeigen, wo Nachrichten im Prozess erwartet werden [19]. Events werden als Kreise mit unterschiedlichen Rändern dargestellt (siehe Abbildung 2.3).



Abbildung 2.3: Start-Event, Ende-Event und Intermediate-Event

2.4 Gateways

Gateways sind ein Tool zum Steuern des Prozessablaufs. Hier können Sequenzflüsse geteilt oder vereint werden. Relevant für diese Arbeit ist lediglich das exklusive Gateway. Dort wird je nach einer gegebenen Bedingung der Sequenzfluss in genau eine von zwei Richtungen weitergeleitet [19]. Ein Gateway wird durch einen Diamanten dargestellt (siehe Abbildung 2.4).

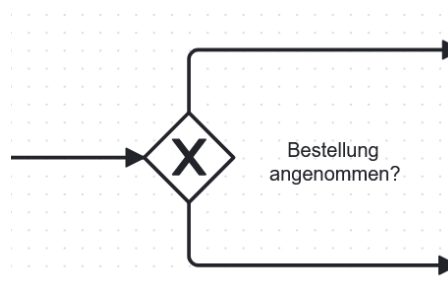


Abbildung 2.4: Exklusives Gateway mit Bedingung

2.5 Datenobjekte und Assoziationen

Mithilfe von Datenobjekten können Informationen im Laufe des Prozesses geladen und gespeichert werden. Assoziationen ermöglichen es, Informationen zwischen Datenobjekten und Aktivitäten zu transportieren. Sie haben keinen Einfluss auf den Prozessablauf. Datenobjekte werden durch eine Notiz und Assoziationen durch einen gestrichelten Pfeil mit offener Spitze dargestellt (siehe Abbildung 2.5). Die Informationen werden in Richtung des Pfeils transportiert [19].

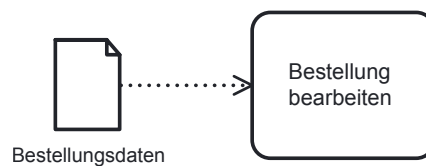


Abbildung 2.5: Datenobjekt mit Assoziation zu einer Aufgabe

2.6 BPMN 2.0 XML

BPMN 2.0 XML ist eine Auszeichnungssprache, welche auf einer von OMG erstellten XML Schema Definition (XSD) basiert [18]. Sie dient dazu, BPMN 2.0 Modelle abzuspeichern und somit einen Austausch zwischen unterschiedlichen Anwendungen zu ermöglichen. Eine BPMN 2.0 XML Datei hat die Endung *.bpmn* [19].

3 Verwandte Arbeiten

In diesem Kapitel werden relevante Ergebnisse verwandter Werke vorgestellt und aufgezeigt, wie diese Arbeit sie erweitert und wie sie sich von ihnen abgrenzt. Zuerst wird der Begriff der Gamifizierung erklärt und seine zentralen Charakteristiken dargestellt. Danach werden die in dieser Arbeit umgesetzten Gamifizierungselemente erläutert, welche auf den verwandten Arbeiten aufbauen. Anschließend wird thematisiert, wie BPMN-Modelle um Gamifizierungselemente erweitert werden können.

3.1 Gamifizierung

Gamifizierung bezeichnet den "Prozess der spielerischen Gestaltung von Aktivitäten in einem spielfremden Kontext durch die Verwendung von Spiel-Design Elementen" [21] – in dieser Arbeit weiter bezeichnet als Gamifizierungselemente. Durch diese Elemente soll die Motivation, das Engagement und die Freude des Benutzers gefördert werden. Das hat sich in vielen Bereichen als wirksam erwiesen, insbesondere für Unternehmen, die Gamifizierung nutzen, um die Mitarbeiterbeteiligung in ihren Unternehmensprozessen zu steigern [11].

Trotz der vielen Vorteile sollte Gamifizierung jedoch auch mit Vorsicht betrachtet werden. Ein zu hoher Grad an Gamifizierung kann dazu führen, dass der Benutzer von der tatsächlichen Aufgabe abgelenkt wird. Daher ist es wichtig, die spielerischen Elemente in einem adäquaten Gleichgewicht zu wählen [22].

Mit der Frage, wie gute Gamifizierungselemente aussehen, hat sich Kapp in seinem Buch [10] auseinandergesetzt. Seiner Meinung nach beinhalten typische Gamifizierungselemente: *Ziele, Zeit, Belohnungen, Level, Feedback, Wettbewerb, Regeln, Konflikte, Kooperationen, Narrativ und Ästhetik*. Außerdem schreibt er, dass Spiele, oder im Kontext dieser Arbeit Unternehmensprozesse, *wiederholbar* sein sollen.

Die folgenden Motivationstheorien aus [11] bieten ebenfalls wertvolle Einblicke, wie Gamifizierung effektiv gestaltet werden kann.

Die *Goal-Setting-Theorie* hebt die Wichtigkeit von klar formulierten und anspruchsvollen Zielen hervor, welche ein Schlüsselement für die Steigerung der Motivation darstellen. Dies verdeutlicht die Notwendigkeit für Klarheit und Herausforderungen als Prinzipien für die Gamifizierungselemente. Die *Flow-Theorie* auf der anderen Seite fordert ein gutes Gleichgewicht zwischen den Fähigkeiten des Benutzers und der Schwierigkeit der Aufgaben. Demnach ist es wichtig bei der Modellierung darauf zu achten, dass die gestellten Herausforderungen zwar anspruchsvoll, aber nicht zu schwer sind.

Die *Selbstbestimmungstheorie* unterstreicht die Bedeutung von intrinsischer Motivation, welche durch eigenes Interesse entsteht, und zeigt auf, dass es vier Stufen von extrinsischer Motivation gibt, die durch externe Anreize hervorgeht. Die Gamifizierungselemente sollten nach der Theorie so gestaltet werden, dass der Benutzer sowohl aus eigenem Interesse heraus, als auch durch äußere Belohnungen motiviert wird.

In der Literaturanalyse [20], über die Häufigkeit und Auswirkung von Gamifizierungselementen in anderen Arbeiten, die Lehren und Lernen gamifizierten, werden vier Elemente identifiziert, welche besonders geeignet sind: *Punkte*, *Ranglisten*, *Auszeichnungen* und *Herausforderungen*. Zudem wird betont, dass *Feedback-Elemente* vermehrt als Gamifizierungselemente verwendet werden sollten.

In den folgenden Abschnitten werden die vier Gamifizierungselemente vorgestellt, mit denen im Rahmen dieser Arbeit das BPMN-Modell erweitert wird: *Punkte*, *Auszeichnungen*, *Herausforderungen* und eine *Minimap* über den Prozessverlauf als *Feedback-Element*. Zu einem großen Teil decken sie sich mit den Elementen aus [20]. Sie stützen sich zudem auf den von Kapp vorgestellten typischen Gamifizierungselementen [10] und sind gleichzeitig in Übereinstimmung mit den dargelegten Motivationstheorien [11].

Online-*Ranglisten*, welche in [20] als Kernelement identifiziert wurden, würden den Rahmen dieser Bachelorarbeit überschreiten und werden daher nicht berücksichtigt.

3.1.1 Punkte

Als erstes Element zur Gamifizierung werden Punkte als Belohnungssystem eingebaut. Der Benutzer erhält für das erfolgreiche Abschließen von Aufgaben Punkte zur Belohnung [20]. Ob eine Aufgabe erfolgreich abgeschlossen wurde, kann mithilfe der Eingabe des Benutzers bestimmt werden. Die Punkte werden in dieser Arbeit in Form von Erfahrungspunkten oder Münzen verteilt.

Der Benutzer hat ein Level, welches durch erhaltene Erfahrungspunkte ansteigt. Dadurch bekommt der Benutzer ein gutes Gefühl für den eigenen Fortschritt und wird dazu motiviert, sich stärker mit den Prozessen auseinanderzusetzen. Das Levelsystem kann dazu beitragen, den sozialen Aspekt der Gamifizierung zu fördern, indem es einen Rahmen für den Vergleich und freundlichen Wettbewerb zwischen Benutzern bietet [20].

Münzen können innerhalb des Prozesses als eine Art Währung behandelt werden, mit der sich der Benutzer etwa zeitliche oder inhaltliche Erweiterungen, wie einen neuen Versuch für eine nicht erfolgreich abgeschlossene Herausforderung, kaufen kann.

3.1.2 Auszeichnungen

Ähnlichen wie bei den Punkten soll der Benutzer für erfolgreich abgeschlossene Aufgaben oder unter anderen bestimmten Bedingungen, wie bei einem bestimmten eingeschlagenen Pfad im Prozess, Auszeichnungen erhalten können. Diese Auszeichnungen belohnen den Benutzer für besondere Errungenschaften und dienen als einzigartige Sammelobjekte [20]. Jede Auszeichnung kann genau einmal freigeschaltet werden. Der Benutzer bekommt während der gesamten Ausführung des Prozesses Feedback darüber, wie viele Auszeichnungen er im Prozess insgesamt freischalten kann, und welche er bereits freigeschaltet hat.

Das Ziel der Auszeichnungen ist es, den Benutzer dazu zu motivieren, alle Auszeichnungen eines Prozesses freischalten zu wollen, was unter Umständen dazu führt, dass der Prozess mehrfach wiederholt werden muss, um jeden möglichen Pfad zu durchlaufen. Dadurch erkundet der Benutzer den Prozess und setzt sich dadurch zwangsläufig mehr mit dem Prozess und seinem Verlauf auseinander.

3.1.3 Herausforderungen

Herausforderungen stellen das dritte Gamifizierungselement dar und können sich über beliebig viele Aktivitäten erstrecken. Sie sind ein essenzieller Bestandteil von Spielen [20]. Konkret liegt der Fokus in dieser Arbeit auf zeitlichen Herausforderungen, bei denen die zugehörigen Aktivitäten innerhalb einer vorgegebenen Zeit abgeschlossen werden müssen, um eine Belohnung zu erhalten. Die Herausforderungen sorgen für einen abwechslungsreichen Prozessablauf und bieten dem Benutzer zusätzlich Anreize, sich intensiver mit dem Prozess auseinanderzusetzen, die eigenen Fähigkeiten zu verbessern und währenddessen fokussiert zu arbeiten [13].

Der Erhalt von Belohnungen für das erfolgreiche Abschließen der zeitlichen Herausforderung kann zusätzlich noch an weitere Bedingungen geknüpft werden, damit der Benutzer ausschließlich die Belohnung erhält, wenn sowohl die Zeit eingehalten als auch alle Aufgaben korrekt bearbeitet worden sind.

3.1.4 Minimap

Das vierte und letzte Gamifizierungselement ist eine Minimap, die während der Prozessausführung den bisher durchlaufenden Pfad und für die aktuelle Aufgabe die nächsten möglichen Pfade anzeigt. Auf der Minimap können relevante Informationen über die Aufgaben dargestellt werden, wie die Belohnung, die der Benutzer erhält, wenn er die Aufgabe erfolgreich bearbeitet. Auf der Karte wird außerdem dargestellt, welche Aufgaben Teile von Herausforderungen sind. Sie ist von erfolgreichen Lernplattformen und Spielen wie "Duolingo" [4] und "Detroit: Become Human" [3] inspiriert.

Die Minimap liefert dem Benutzer eine visuelle Darstellung des Prozessverlaufs und ermöglicht so ein besseres Verständnis der eigenen Position innerhalb des Prozesses. Genau wie in "Detroit: Become Human" [3] lassen sich durch diese Übersicht unerforschte Bereiche leichter identifizieren und der Fortschritt besser nachvollziehen. Die Karte verdeutlicht zudem, welche Aufgaben einen Einfluss auf den Ablauf des Prozesses haben und welche Veränderungen sie bewirken.

Außerdem ermöglicht die Minimap dem Benutzer, genau wie in "Duolingo" [4], auf einen Blick zu erkennen, an welchen Stellen im Prozess Belohnungen wie Punkte

und Auszeichnungen vergeben werden. Dies steigert die Motivation, sich weiter in den Prozess zu vertiefen und die verschiedenen Aspekte der Gamifizierung zu erkunden. Insgesamt fördert die Minimap somit das Feedback und das Engagement des Benutzers bei der Interaktion mit BPMN-Prozessen.

3.2 Erweiterung des BPMN-Modells

Dieses Kapitel thematisiert die Erweiterung des BPMN-Modells. Ein verwandtes Werk in diesem Bereich ist [17], welches einen Erweiterungsansatz für die Modellierung von Gamifizierungselementen im BPMN-Modell vorstellt.

In der Publikation wird jeder Aktivität ermöglicht, zu einer gamifizierten Aktivität zu werden. Dies wird durch einen entsprechenden Typen, welchen die Aktivität zugewiesen bekommt, deutlich gemacht. Eigenschaften der gamifizierten Aktivität, wie die Bedingungen, unter denen ein Benutzer belohnt wird und welche Belohnung er erhält, werden durch Datenobjekte modelliert. Diese sind durch Assoziationen mit den jeweiligen Aktivitäten verbunden [17]. Die Abbildung 3.1 zeigt ein einfaches Beispiel.

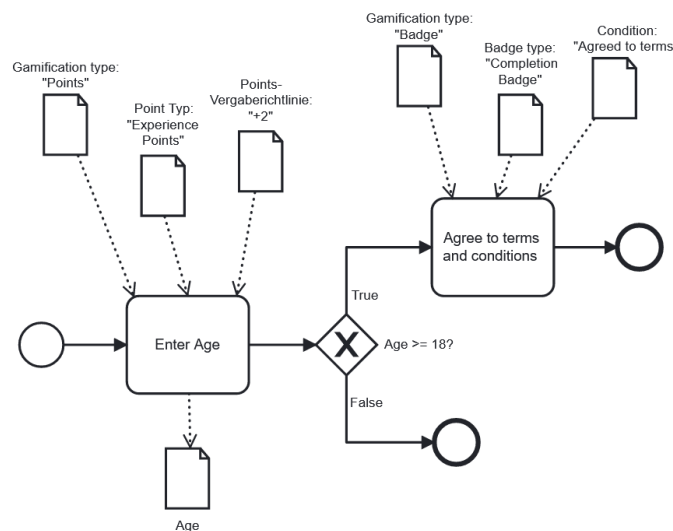


Abbildung 3.1: Beispiel für Gamifizierungselemente in BPMN 2.0

Die Arbeit schließt mit einer Expertenbefragung ab, welche zu dem Schluss kommt, dass die vorgestellte Notationserweiterung das Verständnis von Gamifizierungselementen verbessert und die Modellierung von Gamifizierung in Geschäftsprozessen vereinfacht [17]. Demzufolge bietet die Notationserweiterung eine vielversprechende Methode zur Gamifizierung von Unternehmensprozessen.

Die vorgestellte Erweiterung des BPMN-Modells bietet eine gute Basis für die in dieser Arbeit vorgestellte webbasierte Umgebung. Mit der durch Experten validierten Methodik als Grundlage und den in dem Kapitel 3.1 vorgestellten Gamifizierungselementen, ist es möglich, ein System für die Erstellung und Ausführung von gamifizierten BPMN 2.0 Modellen zu konzipieren und zu implementieren. Diese Arbeit zielt darauf ab, die in [17] vorgestellten Konzepte zu erweitern und eine Plattform bereitzustellen, die sowohl den Modellierungsprozess unterstützt als auch die Ausführung der modellierten gamifizierten Prozesse ermöglicht.

3.3 Abgrenzung zu anderen Arbeiten

Im Gegensatz zu den anderen vorgestellten Arbeiten konzentriert sich diese Arbeit auf eine spezifische, technologieorientierte Perspektive. Sie geht über die konzeptionelle Betrachtung von Gamifizierungselementen in einem BPMN-Modell hinaus und setzt diese in einer webbasierten Umgebung praktisch um. Dies ermöglicht die tatsächliche Erstellung und Ausführung von gamifizierten Prozessen, ein Aspekt, der in bisherigen Arbeiten weniger beachtet wurde.

4 Anforderungsanalyse

Dieses Projekt ist grundsätzlich in zwei Hauptkomponenten unterteilt: den Editor und die Engine. Der Editor wird zur Modellierung der Prozesse genutzt, während die Engine für die Ausführung der Prozesse zuständig ist.

Die Entscheidung, einen eigenen Editor zu entwickeln, resultierte aus mehreren Überlegungen. Zum einen war die Bereitstellung einer intuitiven, benutzerfreundlichen Oberfläche von entscheidender Bedeutung, um dem Benutzer zu ermöglichen, alle für die Ausführung erforderlichen Informationen zu modellieren. Da bereits vorhandene BPMN-Editoren keine direkte Möglichkeit bieten, Gamifizierungselemente in den Prozess einzubinden, konnte diese Anforderung mit vorhandener Software nicht erfüllt werden.

Zweitens war es wichtig, eine Lösung bereitzustellen, die nahtlos in die gesamte Architektur der Umgebung integriert ist. Die Verwendung eines eigenen Editors erlaubt eine direkte Kontrolle und Anpassungsfähigkeit, die bei der Verwendung von Drittanbieter-Software nicht gegeben wäre. Die Implementierung konnte so auf die spezifischen Anforderungen der Gamifizierungselemente abgestimmt werden.

In diesem Kapitel werden die funktionalen Anforderungen an den Editor und die Engine dargelegt. Diese Anforderungen bilden die Grundlage für eine funktionsreiche und leistungsstarke Umgebung zum Modellieren und Ausführen von gamifizierten BPMN Modellen. Jede funktionale Anforderung besteht aus einer ID, einem Titel, einer Beschreibung und möglichen Referenzen zu anderen Anforderungen. Dadurch wird eine präzise Beschreibung der Software ermöglicht. Dieses Kapitel ist ein wichtiger Bestandteil der Arbeit, da es das Ziel der Implementierung klar definiert und den Grundstein für einen messbaren Implementierungserfolg bildet.

4.1 Funktionale Anforderungen an den Editor

In den folgenden Tabellen sind die funktionalen Anforderungen an den Editor aufgelistet, die essenziell für ein optimales Arbeitsergebnis und eine erfolgreiche Umsetzung sind.

ID:	FAED001
Titel:	Unterstützte BPMN Elemente
Beschreibung:	Der Editor unterstützt alle für diese Arbeit relevanten BPMN Elemente. Das sind: • Start-Event • Ende-Event • Intermediate-Event mit Optionen für ein Gamifizierungsevent • Aktivität • Gateway
Abhängigkeit:	FAED005

ID:	FAED002
Titel:	Weitere unterstützte Elemente
Beschreibung:	Der Editor unterstützt neben den klassischen BPMN Elementen noch zusätzliche Elemente, welche beim Export zu BPMN Elementen transformiert werden: • Herausforderung, welche wie eine BPMN Gruppe funktioniert und Optionen für ein Gamifizierungsevent hat, das bei erfolgreichem Abschluss eintritt. • Info-Event, dem man eine Nachricht mitgeben kann, die dem Benutzer angezeigt werden soll.
Abhängigkeit:	FAED005

ID:	FAED003
Titel:	Modellieren von Prozessen
Beschreibung:	Mit dem Editor können BPMN konforme Prozesse modelliert werden. Dafür müssen die BPMN Elemente platziert und korrekt mit Sequenzflüssen verbunden werden können.
Abhängigkeit:	FAED001, FAED002

ID:	FAED004
Titel:	Funktionsumfang der Aktivität
Beschreibung:	Eine Aktivität verfügt über: • Ein Eingabefeld für den Titel der Aufgabe. • Eine Auswahl aller Aktivitätstypen (Texteingabe, Single Choice, Multiple Choice) und den dazugehörigen Auswahlmöglichkeiten, falls notwendig. • Die Möglichkeit, einen Variablennamen einzugeben, über welchen die Benutzereingabe später abgerufen werden kann. • Alle Optionen für ein Gamifizierungsevent.
Abhängigkeit:	FAED001, FAED005

ID:	FAED005
Titel:	Optionen für Gamifizierungsevents
Beschreibung:	Die Optionen für ein Gamifizierungsevent bestehen zuerst aus einer Auswahl des Typen der Gamifizierung. Für Punkte gibt es im Anschluss folgende weiterführende Optionen: • Art der Punkte, wobei in dieser Arbeit Erfahrungspunkte und Münzen unterstützt werden. • Wie sollen die Punkte verändert werden: Auf Wert setzen, inkrementieren oder dekrementieren • Um wie viel sollen die Punkte verändert werden Für Auszeichnungen kann ausgewählt werden, welche Auszeichnung freigeschaltet werden soll. Das Gamifizierungsevent kann optional an eine Bedingung geknüpft werden, welche zutreffen muss, damit der Benutzer die Belohnung erhält.
Abhängigkeit:	-

ID:	FAED006
Titel:	Export
Beschreibung:	Das mit dem Editor erstellte BPMN Modell kann als BPMN 2.0 XML exportiert werden, damit es auch in anderen Editoren geladen und bearbeitet werden kann. Die Optionen für Gamifizierungsevents werden in Datenobjekten abgespeichert und an die dazugehörige Aktivität mit einer Assoziation angebunden [17].
Abhängigkeit:	-

ID:	FAED007
Titel:	Laden erstellter Prozesse
Beschreibung:	Von dem Editor erstellte und exportierte BPMN Modelle können wieder geladen werden, um sie weiter zu bearbeiten
Abhängigkeit:	FAED005

4.2 Funktionale Anforderungen an die Engine

In diesem Abschnitt werden die funktionalen Anforderungen an die Engine vorgestellt. Diese Anforderungen sind entscheidend für die korrekte Ausführung und Darstellung der Prozesse und bilden die Grundlage für eine effektive Umsetzung der Gamifizierungselemente.

ID:	FAEN001
Titel:	Laden von Prozessen
Beschreibung:	Die Engine kann die vom Editor erstellten gamifizierten BPMN Modelle laden.
Abhängigkeit:	-

ID:	FAEN002
Titel:	Ausführen von Prozessen
Beschreibung:	Gamifizierte BPMN Modelle können von der Engine ausgeführt werden. Dafür müssen alle Aktivitätstypen (Texteingabe, Single Choice und Multiple Choice, Info) unterstützt werden und bei Gateways muss die Engine in der Lage sein auf Basis der Bedingung die korrekte nächste Aktivität zu starten.
Abhängigkeit:	FAEN006

ID:	FAEN003
Titel:	Verwalten von Variablen
Beschreibung:	Die Benutzereingaben werden dem erstellten BPMN Modell entsprechend in Variablen abgespeichert und können bei Bedarf ausgelesen werden. Außerdem werden die gesammelten Punkte und freigeschalteten Auszeichnungen zu jedem Zeitpunkt verwaltet. Dafür kann die Engine nach jeder abgeschlossenen Aktivität überprüfen, ob der Benutzer eine Belohnung erhält.
Abhängigkeit:	-

ID:	FAEN004
Titel:	Anzeigen von Gamifizierungselementen
Beschreibung:	Die Engine ist in der Lage, dem Benutzer die aktuellen Fortschritte der Gamifizierungselemente anzuzeigen: • Das durch Erfahrungspunkte erreichte Level. • Die aktuelle Anzahl von erhaltenen Münzen. • Informationen während einer Herausforderung, wie etwa, dass sich der Benutzer in einer Herausforderung befindet und wie lange Zeit noch übrig ist. • Eine Karte über den Prozessverlauf wie in dem Kapitel Minimap beschrieben
Abhängigkeit:	FAEN003

ID:	FAEN005
Titel:	Auswerten von Herausforderungen
Beschreibung:	Die Engine ist in der Lage beim Abschluss einer Herausforderung zu bestimmen, ob sie erfolgreich abgeschlossen wurde oder sie fehlgeschlagen ist. Je nach Ergebnis erhält der Benutzer die Belohnung.
Abhängigkeit:	FAEN006

ID:	FAEN006
Titel:	Ergebnis von Bedingungen bestimmen
Beschreibung:	An vielen Stellen im Modell tauchen Bedingungen auf, welche von der Engine überprüft werden müssen. Die Engine ist in der Lage, die modellierten Bedingungen zu überprüfen und falls notwendig die Werte der intern gespeicherten Variablen dafür zu benutzen.
Abhängigkeit:	FAEN003

4.3 Nichtfunktionale Anforderungen

Im Folgenden werden die nichtfunktionalen Anforderungen für die gesamte webbasierte Umgebung, bestehend aus Editor und Engine, aufgelistet. Sie sind entscheidend für eine angenehme Entwickler- und Benutzererfahrung.

ID:	NFA001
Titel:	Benutzerfreundlichkeit
Beschreibung:	Der Editor und die Engine sollen intuitiv und leicht verständlich sein.
Abhängigkeit:	-

ID:	NFA002
Titel:	Wartbarkeit
Beschreibung:	Die Softwarearchitektur soll so gestaltet sein, dass sie leicht gewartet und erweitert werden kann. Dafür sollte sie möglichst modular aufgebaut sein.
Abhängigkeit:	-

ID:	NFA003
Titel:	Portabilität
Beschreibung:	Die beiden webbasierten Anwendungen sollen auf verschiedenen Webbrowsern (Chrome, Firefox und Edge) und Betriebssystemen korrekt funktionieren.
Abhängigkeit:	-

5 Design und Funktionalitäten

In diesem Kapitel wird das Design des BPMN-Editors und der BPMN-Engine vorgestellt. Dabei liegt der Fokus bei dem Editor auf der Darstellung der Flow-Elemente wie Aktivität, Gateways und anderen Komponenten. Bei der Engine werden der allgemeine Aufbau und die einzelnen Gamifizierungselemente wie die verschiedenen Aufgabenfelder, das Levelsystem, Punkte und die Minimap gezeigt. Mithilfe von passenden Abbildungen wird anschaulich dargelegt, wie die Anwendung in der Praxis aussieht und welche Funktionen sie für den Benutzer bereitstellt.

5.1 Editor

Wie in Abbildung 5.1 zu sehen, besteht der Editor aus drei Hauptkomponenten, welche den funktionalen Anforderungen FAED003, FAED006 und FAED007 entsprechen: Die Gestaltung des Layouts beinhaltet eine Auswahlleiste für Flow-Elemente auf der linken Seite. Diese Elemente können per Drag-and-drop auf der mittleren Arbeitsfläche platziert werden, auf der der Prozess modelliert wird. Auf der rechten Seite befindet sich ein Menü, in dem Projekte gespeichert und geladen sowie Prozesse für die Engine ins BPMN-Format exportiert werden können.

Die Flow-Elemente, die im Editor zur Verfügung stehen, sind nach den funktionalen Anforderungen FAED001 und FAED002 gewählt. Im Folgenden werden ihre Funktionalitäten genauer erläutert.

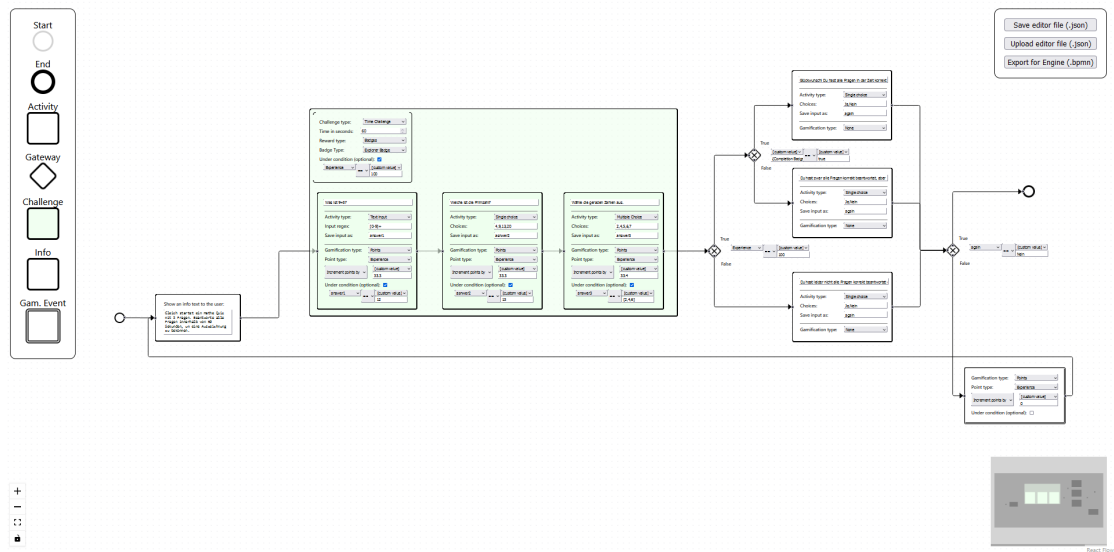


Abbildung 5.1: Vollständiger Editor mit Beispielprozess

5.1.1 Start

Beim Start-Element (siehe Abbildung 5.2) startet der Prozess. Pro Prozess ist in dem Editor genau ein Start-Element erlaubt, welches genau einen ausgehenden Sequenzfluss haben kann. Ist bereits ein Start-Element platziert, wird es in der Auswahlleiste für Flow-Elemente deaktiviert.

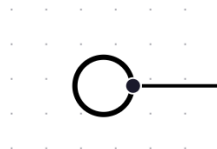


Abbildung 5.2: Start Flow Element im Editor

5.1.2 Ende

Beim Ende-Element (siehe Abbildung 5.3) endet der Prozess. Ein Prozess kann beliebig viele Ende-Elemente beinhalten, welche jeweils beliebig viele eingehende Sequenzflüsse haben können.

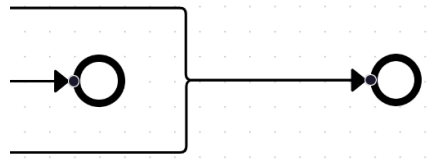


Abbildung 5.3: Ende Flow Elemente im Editor mit unterschiedlich vielen Sequenzflüssen

5.1.3 Aktivität

Für eine Aktivität hat der Benutzer, wie in Abbildung 5.4 zu sehen, alle in der funktionalen Anforderung FAED004 genannten Einstellungsmöglichkeiten. Dabei ist vorgesehen, dass sich der Benutzer von oben nach unten durch die Optionen durcharbeitet, wodurch sich die unteren Eingabefelder je nach Auswahl anpassen, um weiterführende Optionen zu erlauben. Eine Aktivität hat genau einen ausgehenden Sequenzfluss und kann beliebig viele eingehende Sequenzflüsse haben.

Abbildung 5.4: Unausgefüllte und ausgefüllte Aktivität im Editor

Aufgabenstellung

Im Eingabefeld für die Aufgabenstellung der Aktivität (siehe Abbildung 5.5) wird die Aufgabenstellung hineingeschrieben, welche dem Benutzer später in der Engine neben seinen Eingabemöglichkeiten angezeigt wird.

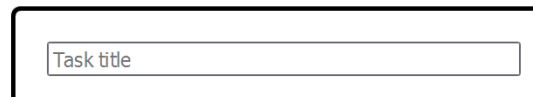


Abbildung 5.5: Titel- und Aufgabenfeld einer Aktivität

Aktivitätstyp

Nach der Aufgabe kann als Nächstes, wie in Abbildung 5.6 zu sehen, der Typ der Aktivität ausgewählt werden. Entsprechend der funktionalen Anforderung FAED004 stehen dafür eine Texteingabe, Single Choice oder Multiple Choice zur Verfügung. Für alle drei Aktivitätstypen können im darunter liegenden Textfeld weitere Optionen, wie die Auswahlmöglichkeiten für den Benutzer, eingetragen werden.

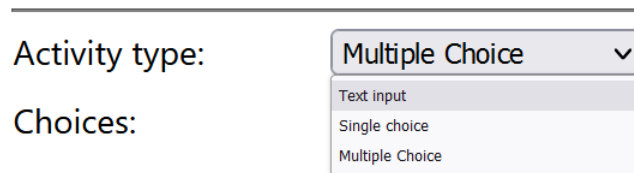


Abbildung 5.6: Auswahlfeld für den Aktivitätstypen

Für die Texteingabe ist es möglich mithilfe von einem regulären Ausdruck (Regex) zu beschränken, welche Eingaben des Benutzers die Engine zulässt, damit die Aufgabe abgeschlossen werden kann. Ein Beispiel für einen Regex für einen einfachen Text ohne Zahlen und Sonderzeichen ist in Abbildung 5.7 zu sehen. Wird kein Regex festgelegt und das Textfeld freigelassen, so wird jede Eingabe von der Engine akzeptiert.

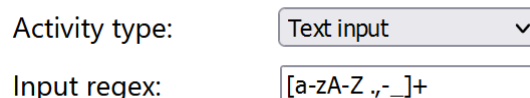


Abbildung 5.7: Beschränkung von Texteingabe mit Regex

Werden Single- oder Multiple Choice als Aktivitätstyp ausgewählt, kann der Benutzer die zur Verfügung stehenden Auswahlmöglichkeiten eintragen (siehe Abbildung 5.8). Dafür werden die einzelnen Möglichkeiten mit Komma getrennt in das Eingabefeld geschrieben.

Activity type:

Choices:

Abbildung 5.8: Auswahlmöglichkeiten für Single- und Multiple Choice

Speichern von Eingaben

Bei jeder Aktivität kann optional die Eingabe des Benutzers für den späteren Gebrauch abgespeichert werden. Dafür muss in das Eingabefeld in Abbildung 5.9 ein selbst gewählter Variablenname eingetragen werden, über den an anderen Stellen im Prozess auf die gespeicherte Eingabe des Benutzers zugegriffen werden kann. Besonders relevant ist diese Funktionalität für Gateways, welche auf Basis von Benutzereingaben den Prozessverlauf steuern können oder für Bedingungen von Gamifizierungsevents, die den Benutzer nur für bestimmte Eingaben belohnen sollen.

Save input as:

Abbildung 5.9: Speichern von Benutzereingaben in einer Variable

Gamifizierungsoptionen

Die Gamifizierungsoptionen einer Aktivität sind optional. Soll die Aktivität gamifiziert werden, stehen genau wie in der funktionalen Anforderung FAED005 beschrieben Punkte und Auszeichnungen als Gamifizierungstypen zur Auswahl (siehe Abbildung 5.10).

Gamification type:

- None
- Points
- Badges

Abbildung 5.10: Auswahlfeld für den Gamifizierungstypen

Werden Punkte als Gamifizierungstyp ausgewählt, kann im Anschluss, wie in Abbildung 5.11 zu erkennen, der Typ der Punkte, die der Benutzer erhalten soll, selektiert

werden. Zur Auswahl stehen hier nach Anforderung FAED005 Erfahrungspunkte und Münzen. Außerdem kann eingestellt werden, um wie viel sich die ausgewählten Punkte bei Erhalt verändern sollen.

The image shows a configuration interface for points. It includes a 'Gamification type' dropdown set to 'Points', a 'Point type' dropdown set to 'Experience', and an 'Increment points by' dropdown set to '[custom value]' with a text input field containing '25'. At the bottom, there is a checkbox labeled 'Under condition (optional):' which is currently unchecked.

Abbildung 5.11: Gamifizierungsoptionen für Punkte

Soll der Benutzer zur Belohnung für die erfolgreiche Bearbeitung der Aktivität eine Auszeichnung erhalten, kann in den weiteren Gamifizierungsoptionen die Art der Auszeichnung ausgewählt werden (siehe Abbildung 5.12). Es gibt bereits vordefinierte Auszeichnungen, jedoch hat der Ersteller des BPMN-Modells auch die Möglichkeit eigene Auszeichnungen mit einem eigenen Namen hinzuzufügen.

The image shows a configuration interface for badges. It includes a 'Gamification type' dropdown set to 'Badges', a 'Badge type' dropdown set to 'Explorer Badge', and a checkbox labeled 'Under condition (optional):' which is currently unchecked.

Abbildung 5.12: Gamifizierungsoptionen für Auszeichnungen

Der Erhalt einer Belohnung kann zusätzlich optional noch an eine Bedingung geknüpft werden. Die Bedingung besteht aus drei Dropdown-Listen, mit welchen die zwei zu vergleichenden Werte und ein Vergleichsoperator ausgewählt werden (siehe Abbildung 5.13). In den Dropdown-Listen der Werte werden alle zur Verfügung stehenden Variablen, inklusive der abgespeicherten Benutzereingaben, angezeigt. Außerdem ist es möglich anstatt einer Variable einen festen benutzerdefinierten Wert einzutragen. Somit können zwei Variablen oder eine Variable mit einem festen Wert verglichen werden.

Werden Eingaben von Multiple Choice Aktivitäten in einer Variable gespeichert, so werden in dieser Variable alle ausgewählten Optionen hinterlegt. Für den Fall, dass

Badge type: Explorer Badge ▾

Under condition (optional): ☒

Experience ▾ == ▾ [custom value] ▾

15

Abbildung 5.13: Bedingung für das Gamifizierungsevent

der Erhalt einer Belohnung bei einer Multiple-Choice-Aktivität von den gewählten Optionen des Benutzers abhängen soll, muss das Vorgehen aus Abbildung 5.14 angewendet werden. Jedes gewünschte Element wird dabei in eckige Klammern geschrieben und mit einem Komma getrennt im Feld [custom value] eingetragen. Im gezeigten Beispiel erhält der Benutzer eine Belohnung, wenn er die Optionen Gelb und Braun auswählt.

Wähle Farben aus

Activity type: Multiple Choice ▾

Choices: Gelb,Rot,Braun,Blau

Save input as: colors

Under condition (optional): ☒

colors ▾ == ▾ [custom value] ▾

[Gelb,Braun]

Abbildung 5.14: Beispiel für Bedingung mit Multiple Choice Eingabe

5.1.4 Gateway

Das Gateway des Editors ist dicht am originalen exklusiven Gateway des BPMN Modells. Es können beliebig viele Sequenzflüsse eingehen und es gibt genau zwei ausgehende Sequenzflüsse. Welcher ausgehende Sequenzfluss beim Ausführen des Prozesses gewählt wird, hängt von der nebenstehenden Bedingung ab. Diese Bedingung funktioniert genauso wie bei den Gamifizierungsoptionen der Aktivität. Um mehr als zwei Ausgänge für eine Bedingung zu erzielen, können wie in Abbildung 5.15 zu sehen mehrere Gateways hintereinander geschaltet werden.

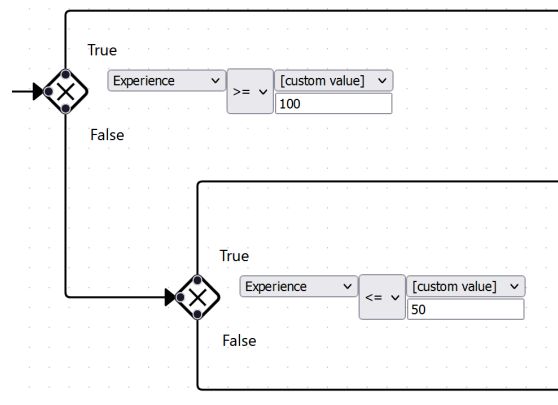


Abbildung 5.15: Zwei hintereinander geschaltete Gateways

5.1.5 Herausforderung

Die Herausforderung ist ein weiteres Gamifizierungselement im Editor. In die platzierte Herausforderung können andere Flow-Elemente, wie Aktivitäten, Gateways und Events hineingezogen werden, um Teil von ihr zu werden. Flow Elemente, welche Teil einer Herausforderung sind, werden mit einer grünen Hintergrundfarbe hervorgehoben. Das Herausforderungselement funktioniert wie eine Gruppe, wird also nicht wie die anderen Flow-Elemente über Sequenzflüsse mit im Prozessverlauf eingebunden. Abbildung 5.16 bildet ein einfaches Beispiel für die Nutzung einer Herausforderung ab. Bei Bedarf kann im Editor die Größe der Herausforderung durch Anklicken und Ziehen angepasst werden.

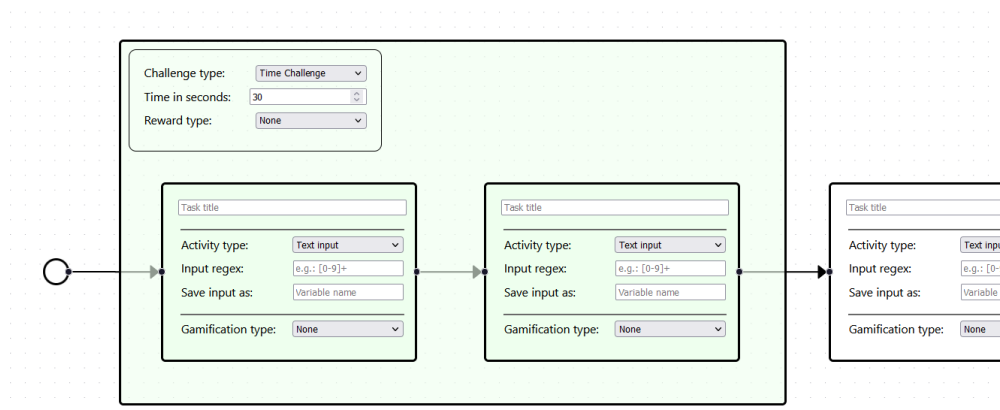
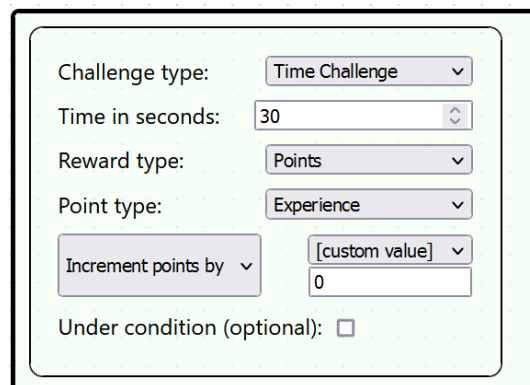


Abbildung 5.16: Beispiel einer Herausforderung im Editor mit Flow Elementen

Oben links in der Herausforderung befinden sich, wie in Abbildung 5.17 zu sehen, die Optionen, in denen die Art der Herausforderung und die Belohnung eingestellt werden können. Wie im Kapitel Gamifizierung dargelegt, beschränkt sich die Auswahl der Art der Herausforderung auf eine zeitliche Herausforderung, für welche die zeitliche Grenze in Sekunden angegeben werden kann. Mit dem Belohnungstypen kann man genau wie bei der Aktivität die Gamifizierungsoptionen einstellen und den Benutzer für erfolgreiches Abschließen der Herausforderung mit Punkten oder Auszeichnungen belohnen.



The image shows a configuration window for a challenge. It contains the following elements:

- Challenge type:** A dropdown menu set to "Time Challenge".
- Time in seconds:** A numeric input field set to "30".
- Reward type:** A dropdown menu set to "Points".
- Point type:** A dropdown menu set to "Experience".
- Increment points by:** A dropdown menu.
- [custom value]:** A dropdown menu set to "[custom value]".
- 0:** A numeric input field set to "0".
- Under condition (optional):** An unchecked checkbox.

Abbildung 5.17: Optionen für eine Herausforderung

5.1.6 Info

Das Info-Element besteht aus einem Textfeld zum Eingeben, was dem Benutzer im Prozess angezeigt werden soll. Mithilfe geschweiften Klammern können Werte von Variablen in den Text eingebaut werden (siehe Abbildung 5.18).

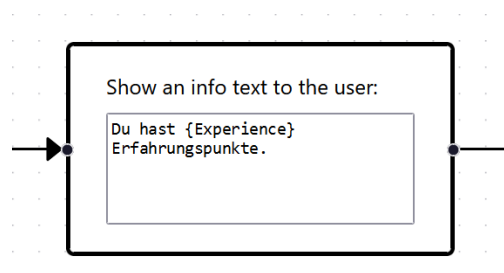


Abbildung 5.18: Info-Element im Editor mit Variable im Text

5.1.7 Gamifizierungsevent

Eine Aktivität kann genau ein Gamifizierungsevent auslösen. Möchte man mehrere Events zwischen zwei Aktivitäten auslösen, kann man das mit dem Gamifizierungsevent Element machen (siehe Abbildung 5.19), welches genau wie ein BPMN 2.0 Intermediate Event funktioniert. Die Optionen sind identisch zu den Gamifizierungsoptionen der Aktivität.

The image shows a configuration dialog for a Gamification Event Element. It has a light gray background and a thin black border. Inside, there are several settings: 'Gamification type:' with a dropdown menu set to 'Points'; 'Point type:' with a dropdown menu set to 'Experience'; 'Increment points by' with a dropdown menu and a text input field containing '0'; and 'Under condition (optional):' with an unchecked checkbox. The dialog is flanked by two horizontal arrows pointing outwards.

Abbildung 5.19: Gamifizierungsevent Element im Editor

5.2 Engine

Die Engine ist in zwei Fenster unterteilt. Beim Starten der Anwendung befindet sich der Benutzer in einem Menü (siehe Abbildung 5.20), in welchem Prozesse hochgeladen und gestartet werden können, was in der funktionalen Anforderung FAEN001 gefordert wird. Beim Start eines Prozesses wird das Ausführungsfenster (siehe Abbildung 5.21) geöffnet. Dort sieht der Benutzer entsprechend der Anforderungen FAEN002 und FAEN004 die aktuelle Aufgabe und alle Gamifizierungselemente.

Gamificated BPMN Engine

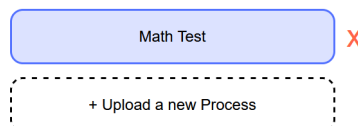


Abbildung 5.20: Hauptmenü der Engine

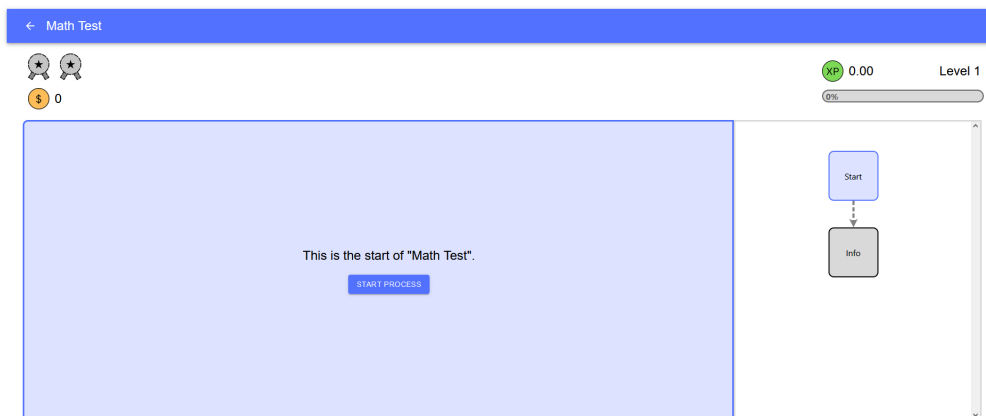


Abbildung 5.21: Ausführungsfenster beim Prozess-Start

5.2.1 Aufgabenfenster

In dem Aufgabenfenster spielt sich die Interaktion des Benutzers mit dem Prozess ab. Hier werden die Aktivitäten und wichtige Informationen über den Prozess angezeigt. Der Abbildung 5.22 ist zu entnehmen, wie die Aktivitäten am Beispiel einer Single Choice Aufgabe und einer Texteingabe aussehen. Weitere Informationen über den Prozess, die dem Benutzer in dem Aufgabenfenster angezeigt werden, sind: *Start des Prozesses*, *Informationen aus der Info-Aktivität*, *Erhalt von Belohnungen*, *Start einer Herausforderung*, *Ende einer Herausforderung* inklusive der erhaltenen Belohnung und *Ende des Prozesses* inklusive zusammenfassende Statistik über alle erhaltenen Belohnungen.

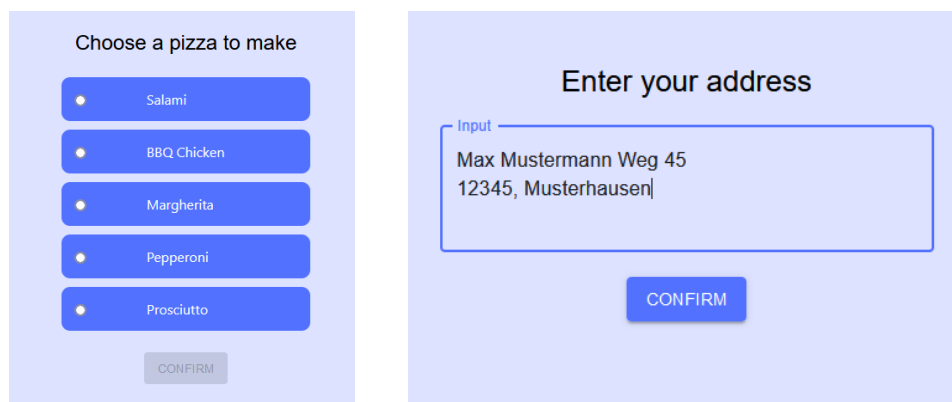


Abbildung 5.22: Beispiele für die Aufgaben im Aufgabenfenster

Durch die Überprüfung der Texteingabe anhand des festgelegten Regex erhält der Benutzer eine Fehlermeldung, falls diese den Anforderungen nicht entspricht. Dadurch werden falsche Eingaben vermieden.

5.2.2 Belohnungen

Ganz oben befinden sich die Anzeigen über die erhaltenen Belohnungen (siehe Abbildung 5.23). Kann eine der drei Belohnungen im gesamten Prozess nicht erhalten werden, wird die jeweilige Anzeige für den Benutzer ausgeblendet. Dadurch ist eindeutig klar, was der Benutzer für Belohnungen erhalten kann.

Für jede freischaltbare Auszeichnung im aktuellen Prozess wird ein Icon einer Medaille angezeigt. Nicht freigeschaltete Auszeichnungen werden grau und freigeschaltete farbig dargestellt. Um zu erfahren, um welche Auszeichnung es sich bei einem der Medaillen handelt, haben alle Icons den Namen als Tooltip hinterlegt.

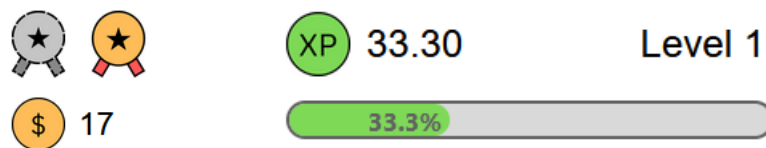


Abbildung 5.23: Auszeichnungen, Münzen und Level

Bei Erhalt einer Belohnung wird der Benutzer im Aufgabenfenster darüber informiert (siehe Abbildung 5.24). Sollte sich der Benutzer währenddessen in einer zeitlichen Herausforderung befinden, wird diese in der Zwischenzeit pausiert. Er hat dann die Möglichkeit, die Belohnung in Ruhe anzuschauen und einzusammeln.

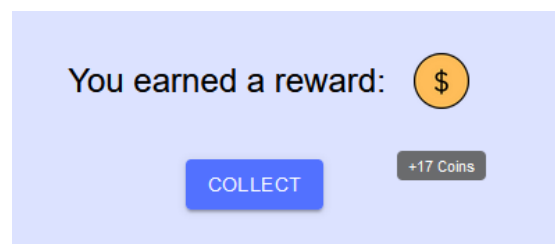


Abbildung 5.24: Erhalt einer Belohnung mit Tooltip des Icons

5.2.3 Herausforderungen

Bevor eine Herausforderung beginnt, wird der Benutzer im Aufgabenfenster über dessen Start informiert, erhält Informationen über die verfügbare Zeit und die potenzielle Belohnung.

Wie in Abbildung 5.25 zu erkennen, kennzeichnet die Engine Herausforderungen durch einen lilafarbenen Hintergrund, der sowohl auf der Minimap als auch am Rand des Aufgabenfensters in Erscheinung tritt. Des Weiteren wird dem Benutzer während einer zeitlichen Herausforderung die verbleibende Zeit anhand eines Countdowns in derselben Farbe angezeigt. Dieser befindet sich direkt über dem Aufgabenfenster.

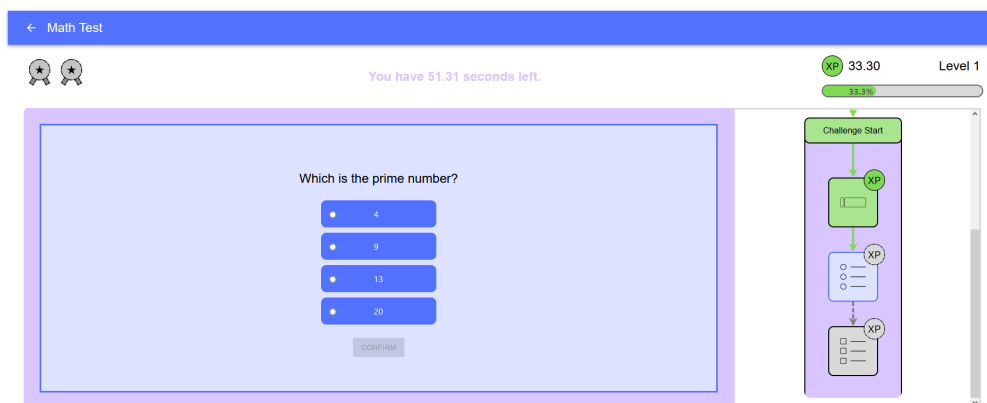


Abbildung 5.25: Ausführungsfenster während laufender Herausforderung

5.2.4 Minimap

Die Minimap befindet sich rechts neben dem Aufgabenfenster und bietet eine übersichtliche Darstellung des Prozesses. Jede Aufgabe und Information, die dem Benutzer im Aufgabenfenster angezeigt wird, ist als abgerundetes Kästchen dargestellt und mit seinen Vorgängern und Nachfolgern über Pfeile verbunden. Wenn eine Aufgabe oder Information in dem Prozess bereits bearbeitet oder angezeigt wurde, wird sie mit einem grünen Hintergrund versehen. Noch nicht besuchte Aufgaben haben einen grauen Hintergrund. Die aktuelle Aufgabe wird durch die gleiche blaue Hintergrundfarbe wie das Aufgabenfenster hervorgehoben.

Die farblichen Hervorhebungen auf der Minimap ermöglichen es, auf einen Blick den Fortschritt im Prozess zu erfassen und unbesuchte Pfade zu identifizieren. Wenn sich der Benutzer in einer Schleife befindet und auf eine bereits bearbeitete Aufgabe trifft, wird diese vor dem Betreten durch einen grünen Hintergrund kenntlich gemacht. Ein visuelles Beispiel dafür ist in der rechten Minimap in Abbildung 5.26 zu sehen.

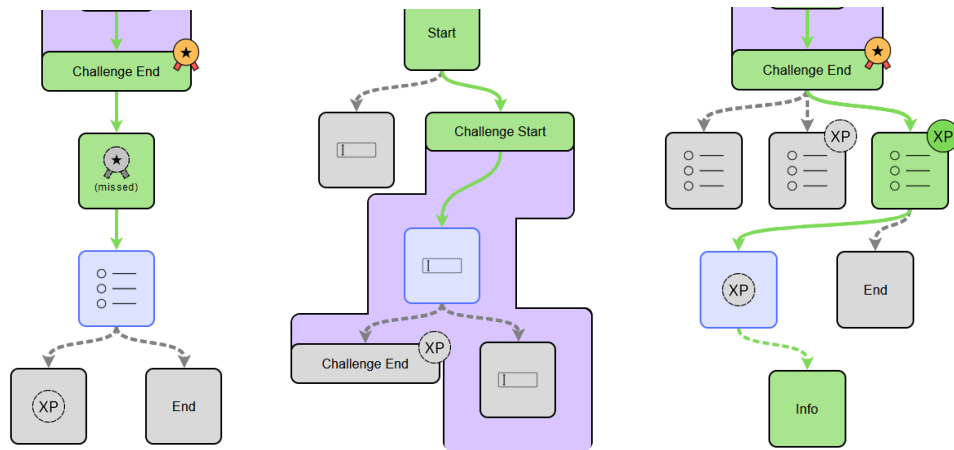


Abbildung 5.26: Diverse Beispiele für die Minimap

Ein wichtiges Merkmal der Minimap ist die Integration der anderen Gamifizierungselemente. Sowohl freischaltbare Belohnungen als auch Herausforderungen werden deutlich gekennzeichnet. In Abbildung 5.26 ist zu sehen, dass freigeschaltete Belohnungen farblich hervorgehoben werden, während nicht freigeschaltete ausgegraut sind.

5.3 Modelltransformation

In [17] wurde bereits eine Möglichkeit aufgezeigt, wie die BPMN 2.0 Notation mit Gamifizierungselementen erweitert werden kann. Das Paper sieht vor, dass Gamifizierungsoptionen mithilfe von Datenobjekten und Assoziationen den jeweiligen Flow Elementen hinzugefügt werden. Im Editor erstellte Prozesse können in dieses Format transformiert werden, um auch mit anderen BPMN Editoren und der Engine

kompatibel zu sein. In Abbildung 5.27 wird dies am Beispiel einer Aktivität demonstriert. Beim Export erhält der Benutzer eine entsprechende BPMN 2.0 XML Datei.

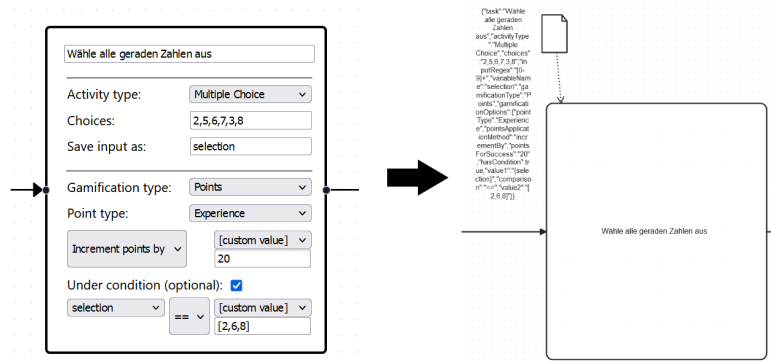


Abbildung 5.27: Transformation vom Editor zu BPMN 2.0 XML

Neben klassischen Flow Elementen bietet der Editor noch die Herausforderungen an, welche beim Export gesondert gehandhabt werden, weil sie kein äquivalentes BPMN Flow Element besitzen. Wie in Abbildung 5.28 zu sehen, werden die Grenzen der Herausforderung, also alle eingehenden und ausgehenden Sequenzflüsse, im Editor beim Export durch Intermediate Events ersetzt, welche den Start und Stopp einer Herausforderung signalisieren. Die Events der Herausforderung bekommen, wie auch andere Flow Elemente, durch Datenobjekte die Gamifizierungsoptionen zugewiesen.

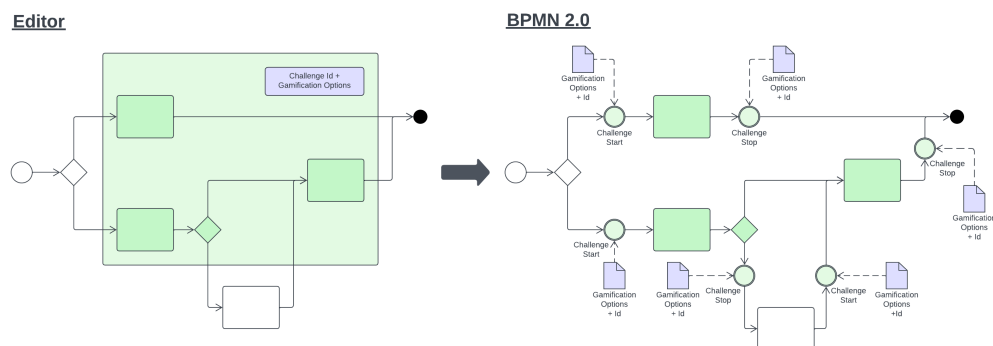


Abbildung 5.28: Transformation von Herausforderungen zu BPMN 2.0 XML

Info-Events werden zu Aktivitäten transformiert, welche den Informationstext als Aufgabenstellung beinhalten.

6 Implementierung

In diesem Kapitel wird die Implementierung der webbasierten Umgebung thematisiert. Dabei werden zuerst die Grundlagen, dann die Implementierung des Editors und zum Schluss die der Engine vorgestellt.

6.1 Grundlagen der Implementierung

Dieser Abschnitt konzentriert sich auf grundlegende Konzepte und Technologien, die für die Implementierung des Editors und der Engine entscheidend waren.

6.1.1 JavaScript

Bei JavaScript handelt es sich um eine objektorientierte, klassenlose und leichtgewichtete Skriptsprache, welche primär im Bereich der Webentwicklung zum Einsatz kommt. Die Sprache wurde 1995 von Netscape entwickelt und ermöglicht eine dynamische Manipulation von Website-Elementen. Der Code wird direkt im Webbrowser ausgeführt und ist dadurch unabhängig von Serverressourcen. JavaScript ist eine dynamisch typisierte Sprache, was bedeutet, dass es bei der Zuweisung von Variablen keine Einschränkung des Typs gibt. Je nach Anwendungsfall ist in JavaScript objektorientierte, funktionale oder prozedurale Softwareentwicklung möglich [12]. Heutzutage ist JavaScript kaum mehr wegzudenken und bildet gemeinsam mit HTML und CSS das Fundament moderner Webentwicklung.

6.1.2 TypeScript

TypeScript wurde 2012 von Microsoft entwickelt und ist eine auf JavaScript basierende Programmiersprache, welche sich durch eine statische Typisierung von ihrem Vorbild unterscheidet. Eine statische Typisierung bedeutet, dass das Programm bereits vor der Ausführung auf die korrekte Nutzung von Datentypen überprüft wird. Situationen, in denen der Entwickler einen Typ erwartet, aber das Programm einen anderen liefert, gelten als eines der häufigsten Probleme beim Programmieren. Gründe hierfür können von unbemerkten Tippfehlern hin zu einem falschen Verständnis von fremdem Code reichen. Durch die Typangaben im Programmcode kann die Fehleranfälligkeit bereits im Vorfeld durch frühzeitiges Erkennen reduziert werden und gleichzeitig steigt die Lesbarkeit, Wartbarkeit und Skalierbarkeit der Software deutlich an [16].

TypeScript wird vor dem Ausführen in nativen Javascript Code transpiliert. Dadurch ist der Code mit jedem modernen Webbrowser und bereits vorhandenen Javascript Bibliotheken und Frameworks kompatibel [16].

6.1.3 Polymorphismus

Polymorphismus, auch als späte Bindung bekannt, ist ein grundlegendes Konzept objektorientierter Programmiersprachen. Es erlaubt verschiedene Objekte, die von einer gemeinsamen Basisklasse erben oder ein gemeinsames Interface verwenden, die gleichen Funktionen zu nutzen. Dabei ist die konkrete Implementierung im jeweiligen Objekt nicht bekannt oder relevant. Dadurch ist es möglich Objekte verschiedener Klassen, welche von einer gemeinsamen Basisklasse erben, in ein Array mit dem Typ der Basisklasse abzuspeichern und auf allen Elementen Funktionen der Basisklasse aufzurufen. Der Code wird durch dieses Konzept modularer und flexibler [9].

In TypeScript kann Polymorphismus durch Klassen, Interfaces und Typen umgesetzt werden. Für diese Arbeit ist die Umsetzung durch Interfaces relevant. Sie ermöglichen es, gemeinsame Strukturen und Methodensignaturen für verschiedene Klassen zu definieren [16].

6.1.4 JSON

JSON (JavaScript Object Notation) ist ein textbasiertes Datenformat, welches zur Darstellung strukturierter Daten verwendet wird. Es wurde ursprünglich von JavaScript abgeleitet und wird mittlerweile sprachunabhängig benutzt. Besonders viel Zuspruch bekommt das Format in Szenarien, in denen Daten zwischen Client und Server ausgetauscht werden, wie in Webanwendungen und APIs [2].

Die Struktur eines JSON Objekts besteht immer aus Schlüssel-Wert-Paaren und geordneten Listen, wobei die Schlüssel immer Zeichenketten sind und die Werte folgende Werte annehmen können: Zeichenketten, Zahlen, Booleans, null, Arrays und Objekte. Ein Objekt ist wiederum genau wie ein JSON Objekt eine ungeordnete Sammlung von Schlüssel-Wert-Paaren. Dadurch können JSON Objekte trotz wenig Regeln eine beliebige Komplexität annehmen [2].

6.1.5 React.js

React.js, häufig auch als React bezeichnet, ist eine offene JavaScript-Bibliothek, welche von Meta Platforms, Inc. (ehemals Facebook) entwickelt wurde. Durch die Kompatibilität von JavaScript und TypeScript kann React.js auch mit TypeScript verwendet werden. Zu den Hauptaufgaben von React gehören die Erstellung und Verwaltung von Benutzeroberflächen in Single-Page Webanwendungen. Mithilfe von React können in Webanwendungen eigene wiederverwendbare Komponenten erstellt werden. Diese Komponenten funktionieren bei der Verwendung wie HTML Elemente und werden durch eine zustandsbasierte Logik verwaltet. Ändert sich der Zustand einer Komponente, so wird diese von React automatisch neu gerendert. Dadurch ist es möglich, komplexe interaktive Webanwendungen effizient und modular zu gestalten [15].

6.1.6 Create-React-App

Create-React-App ist ein Open Source CLI-Tool (Command Line Interface), das von Facebook für eine unkomplizierte Einrichtung und Konfiguration von React Anwen-

dungen entwickelt wurde. Mit nur einem Befehl lassen sich eine initiale grundlegende Projektstruktur, relevante Entwicklungswerkzeuge und deren transitiv abhängige Pakete installieren. Dem Entwickler wird ein bereits lauffähiges Projekt erzeugt, welches ohne weitere Konfiguration mithilfe von mitgelieferten Befehlen ausgeführt oder gebaut werden kann. Durch den Einsatz von Create-React-App kann sich der Entwickler direkt auf das Schreiben von Code fokussieren. Während des Entwicklungsprozesses unterstützt Create-React-App Hot-Reloading, wodurch Änderungen im Code direkt in der parallel laufenden Anwendung angezeigt werden [6].

6.1.7 React Flow

Bei React Flow handelt es sich um eine Open Source Bibliothek für React, mit der man Diagramme, wie Flussdiagramme oder Prozessvisualisierungen, innerhalb einer React Anwendung erstellen, mit ihnen interagieren und diese dynamisch anpassen kann. Die Bibliothek ermöglicht den Entwicklern benutzerdefinierte Kanten- und Knotenelemente zu erstellen, welche in der Anwendung auf einer Arbeitsoberfläche platziert, verbunden und bearbeitet werden können. Außerdem kann der Entwickler eigene Regeln definieren, welche zum Verbinden der Knoten mit Kanten gelten müssen, wodurch die Bibliothek sehr flexibel auf den gewünschten Kontext angepasst werden kann. Die Bibliothek bietet des Weiteren bereits viele nützliche Funktionalitäten wie Drag-and-drop von Knoten, Zoom und Schwenken der Diagrammansicht für die Arbeitsoberfläche an [7].

6.1.8 Zustand

Zustand ist eine leichtgewichtige und skalierbare React Bibliothek zur Zustandsverwaltung in React Anwendungen. Im Vergleich zu Redux, einer der verbreitetsten React Bibliotheken für Zustandsverwaltung, beschränkt sich Zustand auf die wesentlichen Funktionalitäten und bietet daher einen einfachen Einstieg [24].

Arbeitet man lediglich mit React, sind Zustände auf eine lokale Komponente beschränkt. Sie müssen über Parameter (Props) an andere Komponenten übergeben werden, falls diese ebenfalls vom Zustand abhängig sein sollen. [15]. Das sorgt zwar

für eine klare Zugehörigkeit von Zuständen zu Komponenten, sorgt jedoch für vieles vermeidbares Herumreichen von Zuständen, was zu einer erhöhten Komplexität des Codes führt.

Zustand löst dieses Problem durch globale Zustände, welche zentral in einem Zustandsspeicher abgelegt werden. Von diesen Speichern kann es beliebig viele geben, die dann von überall im Projekt aus referenziert werden können. Auf die gleiche Art und Weise können ebenfalls Funktionen in den Zustandsspeichern definiert werden [24].

6.1.9 Weitere Bibliotheken

Zusätzlich zu den zuvor erläuterten Haupttechnologien und -bibliotheken, werden von der Engine noch weitere kleinere Bibliotheken für die Erfüllung spezifischer Aufgaben verwendet. Im Folgenden werden diese Bibliotheken kurz vorgestellt:

- **x2js:** Eine Bibliothek zur Konvertierung von XML-Dateien zu JSON-Objekten und umgekehrt [23]. In der Engine wird sie zum Konvertieren der BPMN-XML-Dateien verwendet.
- **react-xarrows:** Ermöglicht die Erstellung von Pfeilen zwischen React-Komponenten [5], welche in der Minimap benutzt werden.
- **Material UI:** Diese Bibliothek bietet eine Vielzahl von UI-Komponenten für React-Anwendungen. Diese Komponenten folgen den Richtlinien des Google Material Designs [14]. Die Elemente werden für die Gestaltung der Benutzeroberfläche der Engine benutzt.

6.2 Editor

Der Editor wurde in TypeScript und React.js entwickelt und basiert auf der React Flow Bibliothek. Das initiale Projekt wurde mit Create-React-App erstellt. Im Folgenden werden die wichtigsten Konzepte der Implementierung erläutert.

6.2.1 React Flow und JSON-Format

Die `ReactFlow`-Komponente (siehe Quelltext A.1) ist die Basis der React Flow Bibliothek und erzeugt mit allen darin liegenden Komponenten ein interaktives Diagramm, auf dem der Benutzer Prozesse modellieren kann.

Die Bibliothek verwendet ein JSON-Format zur Repräsentation der einzelnen Elemente des Diagramms. Diese Elemente werden als `nodes` und `edges` bezeichnet und stellen die gesetzten Flow-Elemente und Sequenzflüsse dar. Diese Daten werden in einem globalen Speicher mithilfe von Zustand verwaltet (siehe Quelltext A.2) und später beim Export in das BPMN-Format exportiert.

6.2.2 Komponenten zur Repräsentation von Flow-Elementen

Die Arten der Flow Elemente, die innerhalb des Diagramms zur Verfügung stehen, werden durch die `nodeTypes` festgelegt. Diese sind in einem Zustandsspeicher definiert, wie in A.2 gezeigt, und werden in der `ReactFlow`-Komponente benutzt, wie in A.1 dargestellt. Für jedes Flow-Element gibt es eine eigene React Komponente, auf welche in den `nodeTypes` verwiesen wird (siehe Quelltext A.3 für ein Beispiel einer solchen Komponente). Die Komponenten sind alle analog aufgebaut.

Jedes Flow-Element besitzt einen bestimmten Datentyp, der angibt, welche Daten es speichert. Die gespeicherten Daten sind äquivalent zu den Einstellungen, die der Benutzer im Editor treffen kann. So speichert beispielsweise das Gamifizierungs-event (siehe Quelltext A.4) Informationen über die Art der Gamifizierung und die passenden Gamifizierungsoptionen.

Die Komponenten der Flow Elemente verwalten ihre Daten zunächst als lokale Zustände. Bei Veränderungen werden diese Daten an den entsprechenden Knoten in der globalen `nodes`-Liste über die Methode `updateNodeData` weitergegeben (siehe Quelltext A.3), um für einen möglichen Export bereit zu sein.

Mit der Handle Komponente von `ReactFlow` werden die Verbindungspunkte für Sequenzflüsse modelliert.

Die Benutzeroberflächen der Flow-Element-Komponenten werden mithilfe eines dafür erstellten `OptionsContainer` erstellt. Um eine möglichst hohe Modularität zu

gewährleisten, wird dieser Container mit vorgefertigten Komponenten für wiederkehrende Arten von Eingabefeldern befüllt (siehe Quelltext A.3).

Insgesamt ermöglicht die globale Zustandsverwaltung und die modulare Gestaltung der Komponenten eine effiziente Erweiterbarkeit und Wartbarkeit des Editors.

6.3 Engine

Die Engine wurde ebenfalls mit React.js und TypeScript als Webanwendung umgesetzt und es wurde wieder Create-React-App für die Initialisierung verwendet. Im Folgenden werden die Kernkonzepte der Implementierung erläutert.

6.3.1 Prozess-Import und Engine-Architektur

Während des Imports eines Prozesses in der Engine, wird eine sogenannte `NodeMap` (siehe Typdefinition 6.1) erstellt. Diese Map beinhaltet alle Flow-Elemente als Instanzen des Interface `BasicNode`. Neben den Flow-Elementen selbst, speichert die `NodeMap` zusätzlich Informationen darüber ab, welche Flow-Elemente im Prozessablauf auf das jeweilige Element folgen.

Die Beziehungen zwischen dem aktuellen Flow-Element und seinen nachfolgenden Flow-Elementen werden durch einen sogenannten `NextNodeKey` Enum charakterisiert. Der `NextNodeKey` kann drei mögliche Werte annehmen:

- `FALSE`: Kennzeichnet den Pfad, der bei Nichterfüllung einer Bedingung genommen wird.
- `TRUE`: Kennzeichnet den Pfad, der bei Erfüllung einer Bedingung genommen wird.
- `ONLY`: Wird verwendet, wenn nur ein nachfolgendes Flow-Element existiert.

Diese Information ist insbesondere bei Gateways von Bedeutung, die zwei nachfolgende Flow-Elemente besitzen.

Listing 6.1: NodeMap Typdefinition

```
1 export type NodeMap = Map<NodeMapKey, NodeMapValue>
2 export type NodeMapKey = string // Id of flow element
3 export type NodeMapValue = {
4   node: BasicNode,
5   next: NodeMapNext
6 }
7 export type NodeMapNext =
8   Record<NextNodeKey, NodeMapKey> | null
```

Diese Struktur ermöglicht eine effiziente und übersichtliche Repräsentation der Prozessabläufe und hilft bei der Verwaltung und Ausführung der Flow-Elemente innerhalb der Engine.

Die Abbildung 6.1 illustriert die Struktur der Flow-Elemente, die in der NodeMap gespeichert sind. Jedes Flow-Element wird durch eine spezielle Klasse repräsentiert und implementiert das Basisinterface `BasicNode`. Durch Polymorphismus kann die NodeMap jedes dieser speziellen Flow-Elemente aufnehmen und es ist sichergestellt, dass jedes Element die Attribute und Funktionalitäten des `BasicNode` Interface bereitstellt.

Ein zentraler Aspekt des `BasicNode` Interface stellt die `run()`-Methode dar. Sie gibt entweder eine darstellbare React Komponente zurück oder führt eine Operation ohne Rückgabewert aus. Genaue Anwendungen und die Funktionsweise dieser Methode werden im Abschnitt Prozessausführung näher erläutert.

Jedes Flow-Element besitzt zudem die im Editor festgelegten Daten, welche auch Gamifizierungsoptionen enthalten können. Diese Daten sind essenziell für den Ablauf des Prozesses.

6.3.2 Prozessausführung

Die Ausführung eines Prozesses in der Engine folgt einem strukturierten Ablauf, der mithilfe eines Aktivitätsdiagramms visualisiert werden kann. Abbildung 6.2 zeigt den Prozessablauf in der Engine.

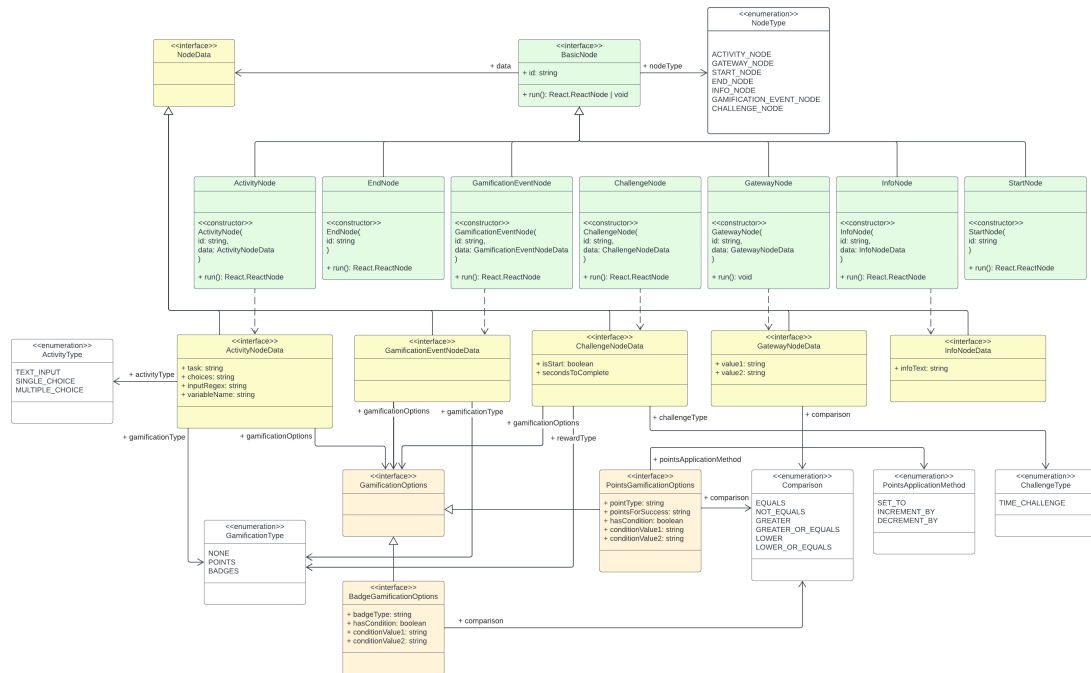


Abbildung 6.1: Architektur der Flow-Elemente mit dem Basisinterface BasicNode

Start des Prozesses

Nach dem Import des Prozesses wird in der NodeMap nach dem Startevent gesucht, welches dann als aktueller Knoten (currentNode) festgelegt wird.

Ausführung des aktuellen Knotens

Die Ausführung eines Knotens beginnt mit dem Aufruf der `run()`-Methode des aktuellen Knotens in der Engine-Komponente (siehe Quelltext A.5). Der Rückgabewert dieser Methode bestimmt das weitere Vorgehen:

- **Rückgabewert vorhanden (ein ReactNode):** Der ReactNode wird dem Benutzer präsentiert. Dies könnte eine Texteingabe, eine Single- oder Multiple Choice Eingabe oder ein Informationstext sein. Anschließend wartet die Engine auf Eingaben und die Bestätigung des Benutzers. In A.6 ist am Beispiel der ActivityNode zu sehen, wie eine konkrete Implementierung der `run()`-Methode mit Rückgabewert aussehen kann.

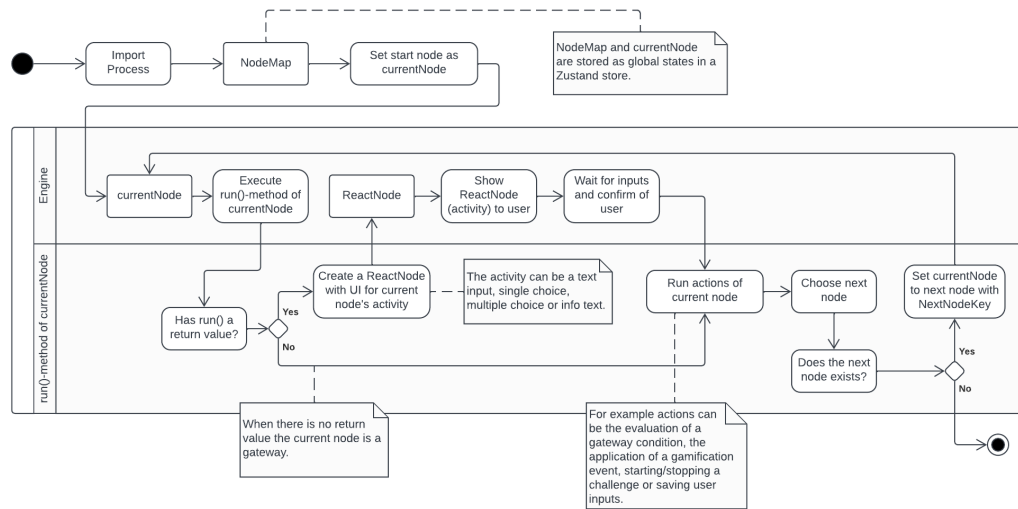


Abbildung 6.2: Aktivitätsdiagramm zum Prozessablauf in der Engine

- **Kein Rückgabewert:** In diesem Fall ist der aktuelle Knoten ein Gateway. Die Engine wählt dann unmittelbar den nächsten Knoten aus, ohne eine Benutzeroberfläche zu präsentieren. Wie eine konkrete Umsetzung der Methode ohne Rückgabewert aussieht, kann A.7 am Beispiel des `GatewayNode` entnommen werden.

Unabhängig vom Rückgabewert der `run()`-Methode werden die Aktionen des aktuellen Knotens ausgeführt, die in der speziellen Implementierung der Methode vorgesehen sind, bevor der nächste Knoten ausgewählt wird. Bei diesen Aktionen handelt es sich beispielsweise um die Auswertung einer Gateway-Bedingung, die Anwendung eines Gamifikationsevents, das Speichern von Benutzereingaben in den dafür vorgesehenen Variablen oder das Starten beziehungsweise Stoppen einer Herausforderung.

Auswahl des nächsten Knotens

Die Engine wählt den nächsten Knoten basierend auf den in der `NodeMap` gespeicherten `NextNodeKey`-Beziehungen. Bei Gateways wird je nach Ergebnis der Bedingung der `TRUE` oder `FALSE` Schlüssel benutzt, um den nächsten Knoten auszuwählen. Bei allen anderen Flow Elementen gibt es nur einen Nachfolger, weshalb `ONLY`

benutzt wird. Der aktuelle Knoten (`currentNode`) wird dann auf den ausgewählten nächsten Knoten gesetzt.

Prüfung auf Ende des Prozesses

Die Engine prüft, ob ein nächster Knoten existiert. Wenn kein nächster Knoten existiert, ist der Prozess abgeschlossen und die Ausführung endet. Andernfalls wird die `run()`-Methode des neuen aktuellen Knotens aufgerufen und der Prozessablauf wiederholt.

6.3.3 Speichern und Anzeigen von Gamifizierungselementen

In der Engine wird ein globaler Zustandsspeicher verwendet, welcher mit der Bibliothek Zustand realisiert wird, um alle relevanten Daten während der Ausführung eines Prozesses zu speichern. Dazu gehören sowohl Benutzereingaben als auch aktuelle Werte der Gamifizierungselemente wie Punkte, Level und freigeschaltete Auszeichnungen. Der Zugriff auf diese Daten ist von jeder Stelle in der Anwendung möglich.

Zur Darstellung der Gamifizierungselemente werden eigenständige React-Komponenten verwendet, welche in der Engine-Komponente (siehe Quelltext A.5) verwendet werden. Eine Änderung der Werte im globalen Zustandsspeicher führt automatisch zu einer Aktualisierung dieser Komponenten. Die Engine umfasst Elemente wie Levelleiste, Münzanzeige, Dialog für freigeschaltete Auszeichnungen, Anzeige der restlichen Zeit während einer Herausforderung und eine Minimap.

Zusammengefasst ermöglicht diese Architektur eine effiziente Speicherung und konsistente Darstellung der Gamifizierungselemente sowie der Benutzereingaben während der Prozessausführung.

7 Ergebnisse und Diskussion

In diesem Kapitel werden die Ergebnisse der Implementierung mit den Anforderungen an den Editor und die Engine verglichen, sowie die Effektivität der ausgewählten Gamifizierungselemente diskutiert.

7.1 Vergleich funktionaler Anforderungen

Den Tabellen 7.1 und 7.3 ist zu entnehmen, dass nahezu alle funktionalen Anforderungen umgesetzt worden sind. Lediglich die FAED007 (Laden von Prozessen) wurde nicht vollständig wie vorher geplant implementiert.

Der Editor unterstützt das Speichern von Prozessen in zwei unterschiedliche Dateiformate: in das JSON-Format, mit welchem React-Flow intern arbeitet und auf welchem der Editor basiert und in den BPMN 2.0 XML Standard, wodurch der Prozess in anderen Editoren geöffnet und in der Engine ausgeführt werden kann. Der Editor unterstützt aktuell ausschließlich das Laden von Prozessen in dem eigenen JSON-Format und folglich nur Prozesse, die im Editor erstellt wurden.

Der Grund dafür ist die komplexe Modelltransformation für die Herausforderungen, die aktuell aufgrund von mangelnder Zeit nur in eine Richtung implementiert ist. Die Entscheidung, Herausforderungen im Editor durch ein Element darzustellen, das kein direktes äquivalentes Flow-Element in BPMN 2.0 besitzt, hat zwar die Benutzerfreundlichkeit beim Modellieren erhöht, im Nachhinein hat sich diese Entscheidung jedoch in Bezug auf die Kompatibilität mit Drittanbieter-Software als sehr arbeitsintensiv herausgestellt.

Tabelle 7.1: Ergebnisse funktionaler Anforderung des Editors

ID	Titel	Ergebnis
FAED001	Unterstützte BPMN Elemente	Implementiert
FAED002	Weitere unterstützte Elemente	Implementiert
FAED003	Modellierung von Prozessen	Implementiert
FAED004	Funktionsumfang der Aktivität	Implementiert
FAED005	Optionen für Gamifizierungsevents	Implementiert
FAED006	Export	Implementiert
FAED007	Laden erstellter Prozesse	Teilweise implementiert

Tabelle 7.2: Ergebnisse funktionaler Anforderung der Engine

ID	Titel	Ergebnis
FAEN001	Laden von Prozessen	Implementiert
FAEN002	Ausführen von Prozessen	Implementiert
FAEN003	Verwalten von Variablen	Implementiert
FAEN004	Anzeigen von Gamifizierungselementen	Implementiert
FAEN005	Auswerten von Herausforderungen	Implementiert
FAEN006	Ergebnis von Bedingung bestimmen	Implementiert

7.2 Vergleich nichtfunktionaler Anforderungen

Die folgende Tabelle stellt den Stand der nichtfunktionalen Anforderungen dar. Im Anschluss werden die Ergebnisse diskutiert.

Tabelle 7.3: Ergebnisse nichtfunktionaler Anforderung

ID	Titel	Ergebnis
NFA001	Benutzerfreundlichkeit	Größtenteils Erfüllt
NFA002	Wartbarkeit	Erfüllt
NFA003	Portabilität	Erfüllt

7.2.1 Benutzerfreundlichkeit

Um die Benutzerfreundlichkeit zu bewerten, werden die 10 Heuristiken von [8] betrachtet. Diese gelten als Grundprinzipien für das Design von Benutzeroberflächen:

1. **Sichtbarkeit des Systemstatus:** In der Engine ist durch Aufgabenfenster und Minimap immer klar zu erkennen, wo man sich gerade im Prozess befindet. Außerdem erhält der Benutzer ein Feedback über die Belohnungen, die er erhält. In dem Editor sieht der Nutzer zu jeder Zeit den erstellten Prozess und weiß so über den Systemstatus Bescheid.
2. **Übereinstimmung zwischen System und realer Welt:** Die gewählten Aufgabentypen und Belohnungen sind aus der realen Welt bekannt.
3. **Benutzerkontrolle und Freiheit:** In der Engine können Antworten in Aufgaben aufgrund von den Gamifizierungselementen nicht rückgängig gemacht werden. Der Benutzer hat aber zu jeder Zeit die Möglichkeit, den Prozess zu verlassen und neu zu starten. Im Editor ist *ESC* unterstützt, aber um die Benutzererfahrung noch zu verbessern, wäre es sinnvoll Aktionen rückgängig machen zu können.
4. **Konsistenz und Standards:** Die Systeme sind konsistent in ihren Gestaltungen. Durch Icons und Farben können Kontexte und Zugehörigkeiten direkt erkannt werden.
5. **Fehlervermeidung:** Die Systeme bieten Hilfestellungen sowohl durch Fehlermeldungen bei falschen Eingaben des Benutzers in der Engine, als auch durch Dropdown-Menüs in dem Editor, die nur gültige Werte zulassen. Auch beim Hochladen von Dateien werden nur passende Dateiformate zugelassen, um Fehler vorzubeugen.
6. **Erkennen statt Erinnern:** Durch wiederkehrende Icons, Farben und bekannten BPMN-Elementen ist die Navigation intuitiv gestaltet und benötigt kein auswendig Lernen des Benutzers.
7. **Flexibilität und Effizienz der Nutzung:** Der Editor erlaubt, Sequenzflüsse ins Leere zu ziehen und über ein auftauchendes Kontextmenü ein neues Flow-Element zu platzieren. Dadurch können erfahrende Benutzer, die die Funktion kennen, schneller Prozesse gestalten. Die Engine bietet keine Abkürzungen, da es durch die Umsetzung der Gamifizierungselemente nicht vorgesehen ist.
8. **Ästhetisches und minimalistisches Design:** Das Design ist nicht mit unnötigen Informationen überladen, sondern beschränkt sich auf die essenziellen Komponenten.

9. **Hilfe Nutzer bei der Erkennung, Diagnose und Behebung von Fehlern:**

Fehlermeldungen über technische Probleme oder das Hochladen von ungültigen Dateien werden bisher lediglich in der Entwicklerkonsole im Webbrowser angezeigt.

10. **Hilfe und Dokumentation:** Da der Editor auf BPMN 2.0 basiert, gibt es für die Erstellung von Prozessen viele hilfreiche Dokumente im Internet zu finden. Für die speziellen Gamifizierungselemente wäre es noch sinnvoll, Hilfestellungen einzubauen. In der Engine wird dem Benutzer zwar in jedem Schritt erklärt, was er zu tun hat, eventuell wäre hier jedoch noch ein Tutorial Prozess möglich, der die Gamifizierungselemente erklärt.

Zusammenfassend kann gesagt werden, dass die Engine und der Editor in vielen Aspekten den 10 Heuristiken von Nielsen [8] entsprechen. Insbesondere punkten sie durch ihre gute Sichtbarkeit des Systemstatus, die hohe Konsistenz und Standards sowie einer ausgeprägten präventiven Fehlervermeidung.

Allerdings wurden auch einige Verbesserungspotenziale identifiziert. Diese liegen vorwiegend in den Bereichen Fehlerbehandlung sowie Hilfe und Dokumentation.

Insgesamt sind die Ergebnisse dieser heuristischen Evaluation positiv und zeigen, dass das System gut gestaltet ist. Um jedoch eine umfassende Beurteilung der Benutzerfreundlichkeit vorzunehmen, sind weitere Untersuchungen wie Benutzertests oder Feldstudien empfehlenswert. Diese könnten die hier identifizierten Verbesserungspotenziale bestätigen oder weitere Aspekte aufdecken, die in dieser Bewertung möglicherweise übersehen wurden.

7.2.2 Wartbarkeit

Im Kapitel Implementierung wird deutlich, dass beim Entwickeln der Software ein großer Fokus auf Modularität und Erweiterbarkeit gelegt worden ist. Mit der gewählten Architektur sind bereits alle notwendigen Komponenten vorhanden, welche für eine effiziente Integration zusätzlicher Gamifizierungselemente notwendig sind.

7.2.3 Portabilität

Die Realisierung des Editors und der Engine als webbasierte Anwendungen ermöglicht eine betriebssystemunabhängige Nutzung. Sie sind mit jedem modernen Webbrowser kompatibel. Lediglich für mobile Geräte sind die Anwendungen nicht ausgelegt, das entsprach jedoch auch nicht der Zielsetzung.

7.3 Effektivität der Gamifizierungselemente

Dieser Abschnitt bietet eine vorläufige Einschätzung zur Effektivität der in dieser Arbeit umgesetzten Gamifizierungselemente. Dabei liegt der Fokus darauf zu ermitteln, ob die Gamifizierung insgesamt als gelungen betrachtet werden kann und welche Elemente besonders zur Motivation beitragen. Die hier getesteten Gamifizierungselemente wurden auf der Grundlage wissenschaftlicher Erkenntnisse ausgewählt, wie sie in [10, 11] dargestellt sind.

7.3.1 Nutzertests

Die Nutzertests wurden anhand eines Mathematik-Quiz durchgeführt, das eine zeitliche Herausforderung beinhaltete. Bei jeder korrekt beantworteten Frage erhielten die Nutzer Erfahrungspunkte. Wenn alle Fragen korrekt beantwortet wurden und die vorgegebene Zeit nicht überschritten wurde, erhielten die Nutzer eine Auszeichnung. Ebenso gab es eine weitere, hinter einer versteckten Bedingung verborgene Auszeichnung. Diese Auszeichnung konnte nur erhalten werden, wenn keine der Fragen im Test korrekt beantwortet wurde. Die Idee an der Stelle war zu überprüfen, ob der Benutzer neugierig wird und den Test wiederholt, um zu versuchen, die Auszeichnung freizuschalten. Für jedes Durchlaufen des Quiz erhielten die Nutzer zudem eine bestimmte Anzahl von Münzen als Vergütung.

In ersten, kleineren Nutzertests konnte bereits eine Tendenz aufgezeigt werden: Die Grundprinzipien der Gamifizierung motivierten die Benutzer dazu, sich intensiver mit einem Prozess auseinanderzusetzen. Sie konnten auf der Minimap noch nicht erkundete Pfade und erreichbare Belohnungen erkennen. Das regte sie dazu an,

diese erkunden und freischalten zu wollen und Aufgaben gegebenenfalls zu wiederholen. Bemerkenswert ist, dass alle teilnehmenden Benutzer das Quiz mindestens einmal wiederholten, um die Auszeichnung freizuschalten, die an eine versteckte Bedingung geknüpft war. Es wurde jedoch auch festgestellt, dass einige Benutzer die Bedingung nicht schnell genug erkannten und daraufhin aufgegeben haben.

Im Kontext der durchgeführten Tests zeigte sich, dass die Münzen ohne unmittelbare Verwendungsmöglichkeit an Attraktivität verloren. Sie traten gegenüber anderen Belohnungsformen in den Hintergrund, da sie über keine zusätzliche Funktionalität verfügten und demzufolge auch keine gesteigerte Motivation bei den Benutzern auslösten. Es wäre jedoch denkbar, dieses Problem zu lösen, indem die Münzen gezielt in die Prozessgestaltung integriert werden. Beispielsweise könnte den Benutzern die Option gegeben werden, während der Prozesse bestimmte Aktionen mit den Münzen auszuführen. Eine weitere Möglichkeit könnte das Anlegen eines zentralen Shops sein, in dem die über mehrere Prozesse hinweg gesammelten Münzen eingelöst werden können. Dadurch würden die Münzen einen konkreten Nutzen bekommen, was ihren Einfluss auf die Motivation des Benutzers verstärken könnte.

7.3.2 Langzeitmotivation

In [22] wird aufgezeigt, dass klassische Gamifizierungselemente wie Punkte und Auszeichnungen möglicherweise nicht auf Dauer die Motivation des Benutzers steigern können. Nach einiger Zeit gewöhnt sich der Benutzer an die Elemente und kann sie womöglich langfristig eher als störend empfinden. Inwiefern sich die Ergebnisse aus [22] in der hier entwickelten Umgebung widerspiegeln, kann lediglich mit einer länger anhaltenden Studie gezeigt werden, welche im zeitlichen Rahmen dieser Arbeit nicht umsetzbar war.

7.3.3 Vorläufige Schlussfolgerungen

Zusammenfassend kann gesagt werden, dass die Gamifizierungselemente in ersten Nutzertests positive Rückmeldungen erhielten und zu einer intensiveren Auseinandersetzung mit den Prozessen anregten. Allerdings haben diese vorläufigen Tests

auch Bereiche aufgezeigt, in denen Verbesserungen oder Ergänzungen sinnvoll wären, um die Gamifizierungselemente noch effektiver zu gestalten. Insbesondere bei den Münzen, welche in den getesteten Prozessen keinen direkten Nutzen hatten. Ebenso stellte sich heraus, dass das Verbergen von Bedingungen der Belohnungen zu Frustration kann, wenn diese nicht in angemessener Zeit erfüllt werden. Eine vorstellbare Lösung wäre die Integration von Hinweisen für den Benutzer, wenn dieser über einen längeren Zeitraum keine Fortschritte erzielt

Es ist wichtig zu betonen, dass diese Ergebnisse auf vorläufigen, kleineren Tests basieren. Eine umfassendere und langfristige Untersuchung ist notwendig, um die Effektivität der Gamifizierungselemente sowie deren langfristige Auswirkungen auf die Motivation und Zufriedenheit der Benutzer zu untersuchen. Die Ergebnisse unterstreichen das Potenzial der Gamifizierung, sind jedoch gleichzeitig eine klare Empfehlung für zukünftige Forschungsarbeiten in diesem Bereich.

8 Aussicht und Zusammenfassung

In dieser Arbeit wurde das Konzept der Gamifizierung von Geschäftsprozessen erörtert und darauf aufbauend eine webbasierte Umgebung für die Erstellung und Ausführung von gamifizierten BPMN 2.0 Modellen implementiert. Dabei wurden Punkte, Auszeichnungen, Herausforderungen und eine Minimap als motivierende Gamifizierungselemente integriert. Die Benutzerfreundlichkeit und Wirksamkeit dieser Elemente wurden anhand von Nutzertests evaluiert, welche sowohl positive Aspekte als auch Bereiche für Verbesserungen aufzeigten.

Obwohl die Arbeit eine Reihe von Gamifizierungselementen erfolgreich implementiert und bewertet hat, gibt es noch viele weitere Elemente und Konzepte, die in zukünftigen Arbeiten untersucht und implementiert werden könnten. Darunter fallen unter anderem die Einrichtung von Benutzerprofilen, die die Fortschritte und Errungenschaften der Benutzer speichern, ein Shopsystem, das es Benutzern ermöglicht, ihre verdienten Punkte oder Belohnungen gegen konkrete Vorteile einzutauschen oder Ranglisten, die einen sozialen Wettbewerbsaspekt hinzufügen.

Die Implementierung dieser zusätzlichen Funktionen könnte das Engagement und die Motivation der Benutzer langfristig weiter steigern und zu einer noch besseren Integration von Gamifizierung in BPMN-Prozesse führen.

Zusammenfassend hat diese Bachelorarbeit gezeigt, dass die Integration von Gamifizierungselementen in BPMN-Prozesse machbar und wirkungsvoll ist. Die Arbeit hat konkrete Methoden und Tools zur Implementierung dieser Elemente präsentiert und erste empirische Hinweise auf ihre Wirksamkeit geliefert. Insgesamt trägt die Arbeit dazu bei, das Potenzial von Gamifizierung für die Gestaltung und Ausführung von Geschäftsprozessen zu verdeutlichen und öffnet die Tür für zukünftige Forschungen und Entwicklungen in diesem aufstrebenden Bereich.

Literatur

- [1] Thomas Allweyer. *BPMN 2.0-Business Process Model and Notation: Einführung in den Standard für die Geschäftsprozessmodellierung*. BoD–Books on Demand, 2020. ISBN: 9783750439016.
- [2] Tim Bray. *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC 7159. März 2014. DOI: 10.17487/RFC7159. URL: <https://www.rfc-editor.org/info/rfc7159>.
- [3] Quantic Dream. *Detroit: Become Human*. 2023. URL: <https://www.quanticroom.com/en/detroit-become-human> (besucht am 18.05.2023).
- [4] Duolingo. *Duolingo - Die weltbeste Methode, eine Sprache zu lernen*. 2023. URL: <https://de.duolingo.com/> (besucht am 18.05.2023).
- [5] Eliav2. *react-xarrows*. Official GitHub Repository. 2023. URL: <https://github.com/Eliav2/react-xarrows> (besucht am 19.05.2023).
- [6] Facebook, Inc. *Create React App - Set up a modern web app by running one command*. Official Create-React-Website. 2022. URL: <https://create-react-app.dev/> (besucht am 27.04.2023).
- [7] webkid GmbH. *React Flow - Wire Your Ideas With React Flow*. Official React-Flow Website. 2022. URL: <https://reactflow.dev/> (besucht am 07.06.2023).
- [8] Nielsen Norman Group. *10 Usability Heuristics for User Interface Design*. 2020. URL: <https://www.nngroup.com/articles/ten-usability-heuristics/> (besucht am 07.06.2023).
- [9] H. Herold, B. Lurz, J. Wohlrab, M. Hopf und Pearson Studium. *Grundlagen der Informatik*. 3. aktualisierte Auflage. Pearson Studium - IT. Pearson Deutschland, 2017. ISBN: 9783863268039. URL: <https://books.google.de/books?id=GQf8twEACAAJ>.

- [10] K.M. Kapp. *The Gamification of Learning and Instruction: Game-based Methods and Strategies for Training and Education*. Wiley, 2012. ISBN: 978-1-11-809634-5. URL: <https://books.google.de/books?id=M2Rb9ZtFxccC>.
- [11] Michael Koch, Florian Ott und Stephan Oertelt. *Gamification von Business Software - Steigerung von Motivation und Partizipation*. Forschungsgruppe Kooperationssysteme, Universität der Bundeswehr, Dez. 2013. ISBN: 978-3-943207-07-1.
- [12] Stefan Koch. *JavaScript: Einführung, Programmierung und Referenz - inklusive Ajax*. dpunkt.verlag GmbH, 2011. ISBN: 978-3898647311.
- [13] Richard N Landers und Amy K Landers. „An empirical test of the theory of gamified learning: The effect of leaderboards on time-on-task and academic performance“. In: *Simulation & Gaming* 45.6 (2014), S. 769–785. DOI: 10.1177/1046878114563662.
- [14] Material UI SAS. *Material UI - Overview*. Official Material UI Website. 2023. URL: <https://mui.com/material-ui/getting-started/overview/> (besucht am 02.06.2023).
- [15] Meta Platforms, Inc. *React - A JavaScript library for building user interfaces*. Official React Documentation. 2023. URL: <https://reactjs.org/docs/getting-started.html> (besucht am 26.04.2023).
- [16] Microsoft. *The TypeScript Handbook*. 2023. URL: <https://www.typescriptlang.org/docs/handbook/intro.html> (besucht am 25.04.2023).
- [17] Benedikt Morschheuser, Christian Hrach, Rainer Alt und Christoph Lefanczyk. „Gamifizierung mit BPMN“. In: *HMD Praxis der Wirtschaftsinformatik* 52 (Dez. 2015). DOI: 10.1365/s40702-015-0188-3.
- [18] OMG. *About the Business Process Model And Notation Specification Version 2.0*. 2011. URL: <https://www.omg.org/spec/BPMN/2.0/About-BPMN> (besucht am 20.04.2023).
- [19] OMG. *Business Process Model and Notation (BPMN)*. 2011. URL: <https://www.omg.org/spec/BPMN/2.0/PDF> (besucht am 01.05.2023).

- [20] Mohd Hishamuddin Abdul Rahman, Dr Ismail, Anida Zaria Binti Mohd Noor und Nor Syazwani Binti Mat Salleh. „Gamification elements and their impacts on teaching and learning - A review“. In: *The International Journal of Multimedia & Its Applications (IJMA)* Vol 10 (2018). DOI: 10.5121/ijma.2018.10604.
- [21] Michael Sailer. „Wirkung von Gamification auf Motivation“. In: *Die Wirkung von Gamification auf Motivation und Leistung: Empirische Studien im Kontext manueller Arbeitsprozesse*. Wiesbaden: Springer Fachmedien Wiesbaden, 2016. ISBN: 978-3-658-14309-1. DOI: 10.1007/978-3-658-14309-1_4. URL: https://doi.org/10.1007/978-3-658-14309-1_4.
- [22] Stefan Stieglitz. „Gamification – Vorgehen und Anwendung“. In: *HMD Praxis der Wirtschaftsinformatik* 52.6 (2015). ISSN: 2198-2775. DOI: 10.1365/s40702-015-0185-6. URL: <https://doi.org/10.1365/s40702-015-0185-6>.
- [23] abdolence. *x2js*. Official GitHub Repository. 2023. URL: <https://github.com/abdolence/x2js> (besucht am 08.06.2023).
- [24] pmndrs. *Zustand*. Official GitHub Repository. 2023. URL: <https://github.com/pmndrs/zustand> (besucht am 27.04.2023).

A Quelltexte

In diesem Anhang sind mehrere Quellcode-Ausschnitte aufgeführt.

Listing A.1: Grundstruktur des Editors durch React Flow

```
1 <ReactFlow ref={reactFlowWrapper}
2   nodes={nodes}
3   edges={edges}
4   onNodesChange={onNodesChange}
5   onEdgesChange={onEdgesChange}
6   onConnect={onConnect}
7   onConnectStart={onConnectStart}
8   onConnectEnd={onConnectEnd}
9   onDragOver={onDragOver}
10  onNodeDragStart={onNodeDragStart}
11  onNodeDragStop={onNodeDragStop}
12  onDrop={onDrop}
13  nodeTypes={nodeTypes}
14  selectNodesOnDrag={false}
15  defaultEdgeOptions={{
16    type: "smoothstep"
17  }}
18  deleteKeyCode={["Backspace", "Delete"]}
19 >
20 <Controls />
21 <Background variant={BackgroundVariant.Dots} />
22 <Panel position="top-left">
23   <NodesToolbar />
24 </Panel>
```

```
25     <Panel position="top-right">
26         <ControlsToolbar />
27     </Panel>
28     [...]
29 </ReactFlow>
```

Listing A.2: nodes, edges und nodeTypes als globale Zustände in Zustand

```
1 export const useStore = create<RFState>((set, get) => ({
2     nodes: [],
3     edges: [],
4     nodeTypes: {
5         activityNode: ActivityNode,
6         startNode: StartNode,
7         endNode: EndNode,
8         gatewayNode: GatewayNode,
9         challengeNode: ChallengeNode,
10        infoNode: InfoNode,
11        gamificationEventNode: GamificationEventNode
12    },
13    [...]
14 }));
```

Listing A.3: GamificationEventNode Komponente

```
1 export default function GamificationEventNode({
2     id, selected, data
3 }): NodeProps<GamificationEventNodeData> {
4
5     // all local states for this node
6     const updateNodeData = useStore((state) =>
7         state.updateNodeData
8     )
9     const [gamificationType, setGamificationType] = [...]
10    const [gamificationOptions, setGamificationOptions] =
11        [...]
```



```

12
13 // Update node data
14 useEffect(() => {
15     updateNodeData<GamificationEventNodeData>(id, {
16         // New node data
17         [...]
18     })
19 }, [...])
20
21 return (
22     <div style={{
23         ...eventShapeStyle,
24         [...],
25         borderColor: selected ? selectedColor : undefined
26     }}>
27         { /* Control connectivity of sequence flows */ }
28         <Handle style={handleStyle} type="source"
29             position={Position.Right} />
30         <Handle style={handleStyle} type="target"
31             position={Position.Left} />
32         { /* Create UI for gamification options */ }
33         <OptionsContainer>
34             <DropdownOption
35                 title={ "Gamification type" }
36                 values={ Object.values(GamificationType) }
37                 selectedValue={ gamificationType }
38                 onChange={ [...] }
39             />
40             <GamificationOptions
41                 parentNodeId={ id }
42                 gamificationType={ gamificationType }
43                 gamificationOptions={ gamificationOptions }
44                 onChange={ [...] }
45             />

```

```

46     </OptionsContainer>
47   </div>
48 )
49 }

```

Listing A.4: GamificationEventNodeData

```

1 export type GamificationEventNodeData = {
2   backgroundColor?: string
3   gamificationType? : GamificationType
4   gamificationOptions?: PointsGamificationOptionsData
5     | BadgeGamificationOptionsData
6 }

```

Listing A.5: Implementierung der Engine-Komponente

```

1 export default function Engine() {
2
3   const isProcessReady = useFlowStore((state) =>
4     state.isProcessReady)
5   const currentNode = useFlowStore((state) =>
6     state.currentNode)
7
8   return (
9     <>
10      { isProcessReady && ( [...]
11        <BadgesInfo /> [...]
12        <CoinsInfo /> [...]
13        <ChallengeInfo /> [...]
14        <LevelProgressBar color={"#7ed957"} /> [...]
15
16        { /* Execute the run method of current node
17          everytime it updates */}
18        { currentNode !== null && (
19          currentNode.node.run() || <></>
20        ) }

```

```

21
22         <Minimap />
23     [...] ) }
24 </>
25 )
26 }

```

Listing A.6: Implementierung der run()-Methode am Beispiel der ActivityNode

```

1  run(): React.ReactNode {
2
3      // Two steps activity (1. task, 2. maybe reward)
4      // This ReactNode will be returned at the bottom
5      const ActivityComponent: React.FC<{ data:
6          ActivityNodeData }> = ({data}) => {
7
8          const [confirmed, setConfirmed] = useState(false);
9
10         const onConfirmTask = (input: string | string[]) =>{
11             // save user input as variable in store
12             if (this.data.variableName !== undefined
13                 && this.data.variableName !== "")
14             {
15                 useVariablesStore.getState().setVariable(
16                     this.data.variableName, input
17                 )
18             }
19
20             // Check if the user gets a reward
21             if (getGamificationConditionResult([...])
22                 && !isBadgeAlreadyUnlocked([...])
23             ) {
24                 // Show collect reward activity next (line 33)
25                 setConfirmed(true);
26             } else {

```

```

27      // Skip to next flow element
28      useFlowStore.getState().nextNode()
29    }
30  } // onConfirmTask end
31
32  // return a activity for the user to collect a
33  // reward, if the task was completed successfully
34  if (confirmed) {
35    return React.createElement(CollectRewardActivity, {
36      key: this.id + "collect",
37      gamificationType: [...],
38      gamificationOptions: [...],
39      onCollectClicked: () => {
40        applyGamification([...])
41        // Skip to next flow element
42        useFlowStore.getState().nextNode()
43      }
44    })
45  }
46
47  // Return a task depending on the activity type
48  switch (this.data.activityType) {
49    case ActivityType.TEXT_INPUT:
50      return React.createElement(TextInputActivity, {
51        key: this.id,
52        data: this.data,
53        onConfirm: (input: string) => {
54          this.onConfirmTask(input)
55        }
56      })
57    case ActivityType.SINGLE_CHOICE:
58      return React.createElement(SingleChoiceActivity, {
59        [...] // analog to first case
60      })

```

```
61     case ActivityType.MULTIPLE_CHOICE:
62     return React.createElement(MultipleChoiceActivity, {
63       [...] // analog to first case
64     })
65   }
66 }
67
68 // This is the return statement of the run()-method
69 return React.createElement(ActivityComponent, {
70   key: this.id,
71   data: this.data,
72 })
73 }
```

Listing A.7: Implementierung der run()-Methode am Beispiel des GatewayNode

```
1 run(): void {
2   const flowState = useFlowStore.getState()
3   const {
4     value1, comparison, value2
5   } = this.data
6
7   if (evaluateCondition(value1, comparison, value2)) {
8     flowState.nextNode(NextNodeKey.TRUE)
9   } else {
10    flowState.nextNode(NextNodeKey.FALSE)
11  }
12 }
```

Abbildungsverzeichnis

2.1	Aufgabe ohne und Aufgabe mit Typ	4
2.2	Sequenzfluss zwischen zwei Aufgaben	4
2.3	Start-Event, Ende-Event und Intermediate-Event	5
2.4	Exklusives Gateway mit Bedingung	5
2.5	Datenobjekt mit Assoziation zu einer Aufgabe	6
3.1	Beispiel für Gamifizierungselemente in BPMN 2.0	11
5.1	Vollständiger Editor mit Beispielprozess	21
5.2	Start Flow Element im Editor	21
5.3	Ende Flow Elemente im Editor mit unterschiedlich vielen Sequenz- flüssen	22
5.4	Unausgefüllte und ausgefüllte Aktivität im Editor	22
5.5	Titel- und Aufgabenfeld einer Aktivität	23
5.6	Auswahlfeld für den Aktivitätstypen	23
5.7	Beschränkung von Texteingabe mit Regex	23
5.8	Auswahlmöglichkeiten für Single- und Multiple Choice	24
5.9	Speichern von Benutzereingaben in einer Variable	24
5.10	Auswahlfeld für den Gamifizierungstypen	24
5.11	Gamifizierungsoptionen für Punkte	25
5.12	Gamifizierungsoptionen für Auszeichnungen	25
5.13	Bedingung für das Gamifizierungsevent	26
5.14	Beispiel für Bedingung mit Multiple Choice Eingabe	26
5.15	Zwei hintereinander geschaltete Gateways	27
5.16	Beispiel einer Herausforderung im Editor mit Flow Elementen	27
5.17	Optionen für eine Herausforderung	28
5.18	Info-Element im Editor mit Variable im Text	28

5.19 Gamifizierungsevent Element im Editor	29
5.20 Hauptmenü der Engine	29
5.21 Ausführungsfenster beim Prozess-Start	30
5.22 Beispiele für die Aufgaben im Aufgabenfenster	30
5.23 Auszeichnungen, Münzen und Level	31
5.24 Erhalt einer Belohnung mit Tooltip des Icons	31
5.25 Ausführungsfenster während laufender Herausforderung	32
5.26 Diverse Beispiele für die Minimap	33
5.27 Transformation vom Editor zu BPMN 2.0 XML	34
5.28 Transformation von Herausforderungen zu BPMN 2.0 XML	34
6.1 Architektur der Flow-Elemente mit dem Basisinterface BasicNode . .	43
6.2 Aktivitätsdiagramm zum Prozessablauf in der Engine	44

Listings

6.1	NodeMap Typdefinition	42
A.1	Grundstruktur des Editors durch React Flow	57
A.2	nodes, edges und nodeTypes als globale Zustände in Zustand	58
A.3	GamificationEventNode Komponente	58
A.4	GamificationEventNodeData	60
A.5	Implementierung der Engine-Komponente	60
A.6	Implementierung der run()-Methode am Beispiel der ActivityNode . .	61
A.7	Implementierung der run()-Methode am Beispiel des GatewayNode .	63

Name: Merten Dieckmann

Matrikelnummer: 1058340

Erklärung

Ich erkläre, dass ich die Arbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe.

Ulm, den 15.06.2023, Merten Dieckmann

Merten Dieckmann